

SOUTENANCE ORALE — 30 MINUTES

StudentLink

Script complet de présentation

Plateforme événementielle et sociale pour les étudiants de Clermont-Ferrand

Rubrique	Détail
Candidat	Arthur Gramond
Formation	Concepteur Développeur d'Applications — 2 ^e année, Hesias (Clermont-Ferrand)
Titre visé	Titre professionnel RNCP niveau 6 — Concepteur Développeur d'Applications
Projet	StudentLink — application web mobile-first, développée et déployée en solo
Durée de l'oral	30 minutes de présentation, suivies des questions du jury
Support	Diaporama + démonstration live de l'application en production

Comment utiliser ce document

Ce document n'est pas un texte à réciter mot pour mot : c'est une partition. Les blocs bleus contiennent une formulation prête à l'emploi — apprends-en le sens et les chiffres, pas la ponctuation. Le jour J, tu dois avoir l'air de parler, pas de lire.

Légende des blocs

Bloc	Ce que c'est	Comment le travailler
Objectif	Ce que le jury doit avoir compris à la fin de la partie	Relis-le avant chaque répétition : si le message ne passe pas, la partie est ratée
Bloc bleu	Ce que tu dis, formulé	Apprends les idées et les chiffres exacts ; reformule librement le reste
ATTENTION	Piège, erreur classique ou point sensible	À relire la veille de l'oral
ASTUCE	Effet, démonstration ou formulation qui marque des points	Optionnel, mais fortement recommandé
TRANSITION	La phrase de liaison vers la partie suivante	À connaître par cœur : c'est ce qui donne l'impression de maîtrise

Méthode de répétition conseillée

1. **Passage 1 — lecture à voix haute**, document sous les yeux, sans chronomètre. Objectif : entendre les phrases.
2. **Passage 2 — chronométré**, document sous les yeux. Objectif : caler les minutages de chaque partie.
3. **Passage 3 — plan seul**, uniquement le tableau du plan minuté sous les yeux. Objectif : parler sans script.
4. **Passage 4 — à blanc devant quelqu'un**, avec la démo. Objectif : gérer le stress et les manipulations.
5. **Passage 5 — questions**, quelqu'un te pose les questions de la section « Questions du jury » dans le désordre.

ASTUCE — La partie la plus fragile d'un oral technique n'est jamais le contenu : c'est la démonstration. Répète-la trois fois plus que le reste.

Les chiffres à connaître par cœur

Un jury retient les chiffres. Ceux-ci sont vérifiés dans le projet : tu peux les citer sans risque.

Chiffre	Ce qu'il désigne	Où le placer dans l'oral
42 500	étudiants à Clermont-Ferrand	Partie 1 — contexte local
3 012 800	étudiants en France	Partie 1 — potentiel de déploiement
4	personas construits (3 étudiants + 1 partenaire)	Partie 1 — analyse du besoin
42	fonctionnalités priorisées en MoSCoW	Partie 2 — cahier des charges
7	modules fonctionnels	Partie 2 — structuration du besoin
17	tables relationnelles, normalisées en 3NF	Partie 3 — modèle de données
13	endpoints API JSON (12 dans /api/ + le scan partenaire)	Partie 3 — architecture
31	fonctions PHP dans les quatre fichiers de includes/	Partie 3 — le socle back-end
15 / 10 / 5 / 8	XP par événement / squad / follow / avis	Partie 4 — gamification
8 (+1 prévu)	badges attribués automatiquement	Partie 4 — gamification
5 / 15 min	tentatives de connexion autorisées par IP	Partie 5 — rate limiting
1 heure	durée de validité d'un token de reset	Partie 5 — mot de passe oublié
40 tests / 282 assertions	suite PHPUnit, exécutée en moins de 0,2 s	Partie 6 — qualité
Niveau 5	niveau d'analyse statique PHPStan	Partie 6 — qualité
3	versions de PHP testées en CI (8.1, 8.2, 8.3)	Partie 6 — intégration continue
~2 minutes	délai entre un push Git et la mise en ligne	Partie 6 — déploiement continu
15 → 1	requêtes SQL sur la page Explore avant / après optimisation	Partie 7 — difficultés

Plan minuté de la soutenance

Ce tableau est ton fil conducteur. Imprime-le séparément et garde-le devant toi pendant l'oral.

Minutage	Partie	Points à couvrir
00:00 → 02:00 (2 min)	0. Introduction	Qui je suis · le titre visé · la phrase d'accroche · annonce du plan
02:00 → 06:00 (4 min)	1. Contexte & problématique	Origine personnelle · marché clermontois · problème des deux côtés · problématique · 4 personas · concurrence
06:00 → 09:00 (3 min)	2. Cahier des charges	42 fonctionnalités MoSCoW · 7 modules · exigences non fonctionnelles · périmètre exclu et pourquoi · méthode
09:00 → 14:00 (5 min)	3. Conception	MCD 16 tables · contraintes d'intégrité · cas d'utilisation · séquence d'inscription · architecture MVC léger · baseUrl()
14:00 → 22:00 (8 min)	4. Démonstration	Parcours étudiant · parcours partenaire + analytics · squads & social · gamification · avis · dark mode & PWA
22:00 → 25:00 (3 min)	5. Sécurité	OWASP Top 10 · PDO · bcrypt · session_regenerate_id · CSRF · rate limiting · reset token · headers · contrôle d'accès
25:00 → 28:00 (3 min)	6. Qualité & déploiement	40 tests PHPUnit · PHPStan niveau 5 · CI GitHub Actions · Railway · documentation
28:00 → 30:00 (2 min)	7. Bilan & perspectives	3 difficultés · 4 choix techniques assumés · évolutions · conclusion

ATTENTION — Si tu as 2 minutes de retard au top des 14 minutes, **coupe les personas 2 et 3 (partie 1) et le diagramme de cas d'utilisation (partie 3)**. Ne rogne jamais sur la démonstration : c'est la partie qui prouve que l'application existe.

Ce que contient le document après les sept parties

Section	À quoi elle sert
Annexe A — Le socle back-end	Les 31 fonctions des fichiers includes/, avec pour chacune son rôle et son fonctionnement réel
Annexe B — Les 13 endpoints	Ce que chaque endpoint contrôle, écrit et renvoie
Annexe C — Les pages	Ce que fait chaque page côté serveur, page par page
Annexe D — Six mécanismes	Inscription à un event, check-in, mot de passe oublié, création de compte, abonnement à un étudiant et création d'événement — déroulés étape par étape
Annexe E — Ce qu'il ne faut pas affirmer	Les écarts réels entre le discours et le code, et comment les formuler ou les corriger

Section	À quoi elle sert
Questions du jury	16 questions avec réponse rédigée
Glossaire	15 termes que tu dois savoir définir en une phrase
Checklist et antisèche	La veille, le jour J, et la page unique à garder devant toi

Les annexes A à D ne se récitent pas pendant les 30 minutes : elles servent à la partie 3.6 et surtout aux questions. L'annexe E, elle, doit être lue avant la répétition, parce qu'elle change ce que tu dis.

Partie 0 — Introduction (00:00 → 02:00)

OBJECTIF — Que le jury sache en deux minutes qui tu es, que le projet est réel et non fictif, et où la présentation va l'emmener.

Ce que tu dis

Bonjour, je m'appelle **Arthur Gramond**, je suis en 2^e année de Concepteur Développeur d'Applications à **Hesias**, à Clermont-Ferrand. Je vais vous présenter **StudentLink**, une plateforme web que j'ai conçue et développée seul, de la modélisation de la base de données jusqu'à la mise en production.

Le projet part d'une phrase très simple, que je me suis dite en tant qu'étudiant : **on veut tous se retrouver, mais on veut aussi payer moins cher**. Et aujourd'hui, aucun outil ne répond à ça à Clermont.

Je vais organiser ma présentation en sept temps : le contexte et la problématique, le cahier des charges, la conception, une démonstration de l'application, la sécurité, la qualité et le déploiement, et enfin le bilan.

Points de vigilance

- **Ne commence jamais par la technique.** Le jury doit d'abord comprendre le besoin ; la stack ne veut rien dire tant qu'on ne sait pas à quel problème elle répond.
- **Dis « j'ai conçu et développé », pas « on a fait ».** C'est un projet solo : le jury doit l'entendre explicitement dès la première minute.
- **Reste debout, regarde le jury, pas l'écran.** Les deux premières minutes fixent l'impression générale.
- **Ton écran est déjà prêt :** navigateur ouvert sur la page Explore, aucune fenêtre parasite, notifications coupées.

ATTENTION — Ne t'excuse jamais par avance (« désolé si c'est un peu long », « j'espère que la démo va marcher »). Cela fabrique un doute qui n'existait pas.

TRANSITION → « *Commençons par le contexte, parce que c'est lui qui explique tous mes choix ensuite.* »

Partie 1 — Contexte et problématique (02:00 → 06:00)

OBJECTIF — Prouver que le projet répond à un besoin réel, chiffré et vérifié auprès de personnes réelles — pas à une idée sortie de nulle part.

1.1 — L'origine du projet (45 s)

L'idée ne vient pas d'un sujet imposé, elle vient de mon quotidien. Quand on veut sortir avec mes amis, c'est toujours le même scénario : un groupe WhatsApp qui s'agite, quelqu'un qui cherche sur Instagram, un autre qui demande si c'est payant, un troisième si c'est cher. Et au final, certains restent chez eux — soit parce que l'information est arrivée trop tard, soit parce que le prix les a refroidis.

Le problème, ce n'est pas l'envie de sortir. C'est **l'absence d'un outil pensé pour nous**.

1.2 — Le contexte local et le marché (45 s)

Clermont-Ferrand, c'est **42 500 étudiants**, répartis entre l'UCA, SIGMA Clermont, l'INP, l'IFSI et Hesias. À l'échelle nationale, la France compte **3 012 800 étudiants**.

Le bassin est donc réel, et il est doublement adressable : je commence par Clermont, mais le modèle est **réplicable ville par ville**, parce que rien dans ma modélisation n'est spécifique à Clermont — le champ ville existe déjà sur les établissements.

1.3 — Un problème symétrique (45 s)

Présente le problème des deux côtés : c'est ce qui justifie que l'application ait deux interfaces distinctes.

Le problème est symétrique. **Côté étudiant** : je rate des plans parce que l'information circule mal, et je paie plein tarif alors que des offres étudiantes existent.

Côté établissement : Jean, gérant du Bec qui Pique, ne sait pas combien de personnes vont venir avant 22 heures le soir même. Il communique avec des flyers et des posts Instagram qui se noient dans les fils. Et surtout, il n'a **aucune donnée** sur son public : ni l'âge, ni l'école, ni les horaires d'affluence.

D'un côté une offre qui existe, de l'autre une audience qui ne la trouve pas. C'est ce décalage qui est au cœur du projet.

1.4 — La problématique (30 s)

ATTENTION — C'est LA phrase de l'oral. Articule lentement, marque un temps d'arrêt avant et après. Le jury la note.

D'où ma problématique : **comment permettre aux étudiants de se retrouver facilement, de profiter d'offres exclusives dans les établissements locaux et de s'organiser entre eux — le tout depuis une seule application pensée pour eux ?**

1.5 — Les quatre personas (1 min — 15 secondes chacun)

Ne récites pas les fiches personas : donne pour chacun le profil en trois mots, puis la frustration qui déclenche une fonctionnalité.

Persona	Profil	La phrase à dire	Ce que ça justifie
Arthur	20 ans, Hesias, l'organisateur du groupe, 70-80 € de budget sorties par mois	« Je rate des soirées parce que j'apprends l'événement la veille, voire le soir même. »	Flash Events
Léa	22 ans, M1 Marketing à SIGMA, très active sur les réseaux sociaux	« Les apps type Shotgun ou Time Out sont pensées pour Paris, pas pour Clermont. »	Ancrage local
Lucas	20 ans, INP, arrivé à Clermont cette année, budget serré	« Le running tout seul c'est démotivant, mais je ne sais pas où trouver des gens. »	Squads sportives
Jean	38 ans, gérant de bar, 300-500 € de communication par mois	« Ce que je veux, c'est remplir mon bar le jeudi et savoir d'où viennent mes clients. »	Analytics et check-in

ASTUCE — Enchaîne les quatre personas d'une traite, sans transition entre eux. C'est la partie où tu peux gagner du temps si tu es en retard : garde Arthur et Jean, coupe Léa et Lucas.

1.6 — Le positionnement face à l'existant (15 s)

Face à l'existant : **Shotgun et Dice** sont centrés sur les grandes villes et prennent des commissions trop lourdes pour un bar de quartier. **Facebook Events** n'est plus utilisé par les étudiants. **Instagram** n'a ni inscription, ni gestion de quota, ni données exploitables pour le gérant.

StudentLink se place sur un créneau précis : **local, gratuit pour l'étudiant, et orienté données pour le partenaire.**

TRANSITION → « À partir de ces besoins, j'ai formalisé un cahier des charges. »

Partie 2 — Cahier des charges et périmètre (06:00 → 09:00)

OBJECTIF — Montrer que tu sais cadrer un projet : prioriser, formaliser des exigences non fonctionnelles, et surtout décider ce que tu ne fais pas.

2.1 — Les besoins fonctionnels (1 min 15)

J'ai listé **42 fonctionnalités**, que j'ai priorisées selon la méthode **MoSCoW** — Must have, Should have, Could have — pour être certain de livrer le cœur avant les extras. Elles se répartissent en **sept modules**.

Module	Fonctionnalités principales	Priorité
Authentification	Inscription étudiant et partenaire, connexion, déconnexion, mot de passe oublié, onboarding guidé	MUST
Événements	Explorer avec filtres, consulter le détail, s'inscrire, annuler, Flash Events à compte à rebours, inviter ses abonnés	MUST
Wallet	Pass QR code actifs, historique, cumul des économies, dépôt d'avis après check-in	MUST
Social	Suivre un étudiant, profil public, recherche par nom, école ou centres d'intérêt	MUST
Squads	Créer, rejoindre, quitter, consulter les membres, inviter ses abonnés	MUST
Gamification	Calcul d'XP, niveaux progressifs, déblocage automatique des badges	SHOULD
Partenaire	Créer et gérer ses événements, scanner un QR code, dashboard temps réel, trois graphiques analytiques	MUST
Apparence & accessibilité	Dark mode persistant, PWA installable, navigation clavier, WCAG 2.1 AA	COULD / MUST

2.2 — Les besoins non fonctionnels (45 s)

Côté non fonctionnel, je me suis fixé quatre exigences mesurables.

Performance — moins de deux secondes de chargement sur une connexion 4G, et zéro requête N+1 : toutes les données corrélées sont récupérées par jointure ou par sous-requête.

Mobile-first — la cible principale est **375 à 430 pixels** de large, parce qu'un étudiant qui cherche une soirée sort son téléphone, pas son ordinateur.

Sécurité — couverture explicite de l'**OWASP Top 10**, que je détaillerai dans la partie 5.

Accessibilité — conformité **WCAG 2.1 niveau AA** : contrastes vérifiés, navigation clavier complète, libellés de formulaires associés à leurs champs.

2.3 — Le périmètre exclu, et pourquoi (45 s)

ATTENTION — Ne saute jamais cette partie. Un jury juge autant la capacité à exclure qu'à produire. Sans justification, un manque ressemble à un oubli ; avec justification, il devient un arbitrage.

Ce que j'ai exclu	Ma justification à l'oral
Paieement en ligne (Stripe)	Cela implique une conformité PCI-DSS et un tiers de confiance. Le modèle repose sur la réduction appliquée en caisse, pas sur la billetterie : la valeur était ailleurs.
Application native iOS / Android	J'ai choisi une PWA installable, qui couvre l'essentiel du besoin — icône sur l'écran d'accueil, affichage plein écran — sans passer par les stores ni maintenir deux bases de code.
Géolocalisation temps réel	Coût RGPD élevé (donnée sensible, consentement, durée de conservation) pour une valeur faible : les établissements sont fixes et peu nombreux.
Messagerie instantanée	Fonctionnalité à part entière, qui impose de la modération et un canal temps réel. Les squads et les invitations couvrent déjà le besoin de coordination.
Recommandation par IA	Nécessite un volume de données que la V1 n'a pas encore. J'ai gardé un filtre « pour moi » basé sur l'école et les centres d'intérêt.

2.4 — La méthode de travail (15 s)

J'ai travaillé seul, en itératif, avec des **conventional commits**, un **CHANGELOG versionné** et une intégration continue sur **GitHub Actions** — j'y reviendrai dans la partie qualité. Chaque fonctionnalité a été développée, testée, puis fusionnée une fois la CI au vert.

TRANSITION → « Une fois le besoin cadré, je suis passé à la conception. »

Partie 3 — Conception (09:00 → 14:00)

OBJECTIF — Démontrer la compétence « concevoir » du référentiel : modélisation des données, contraintes d'intégrité, diagrammes UML et architecture applicative justifiée.

ASTUCE — Affiche tes diagrammes, mais ne les lis pas : raconte-les. Un jury décroche instantanément devant quelqu'un qui énumère des noms de tables.

3.1 — Le modèle de données (2 min)

C'est la partie la plus scrutée par un jury CDA. Structure ta réponse en trois temps : les entités centrales, les entités périphériques, puis les contraintes.

J'ai modélisé la base en **17 tables relationnelles, normalisées en troisième forme normale**. Le cœur, ce sont trois entités.

users — avec un champ type en ENUM : étudiant ou partenaire. J'ai fait le choix d'**une seule table utilisateurs** plutôt que deux, parce que l'authentification, le profil et les sessions sont communs aux deux rôles ; ce qui diffère est déporté dans la table établissements. Cela m'évite de dupliquer toute la logique de connexion.

evenements — rattachée à un établissement, avec le quota, la réduction en pourcentage, le prix normal, et deux champs dédiés au Flash : `is_flash` et `flash_expiry`.

inscriptions — c'est la table pivot entre l'étudiant et l'événement. C'est elle qui porte le **QR code** et le **statut**, qui suit un cycle de vie précis : inscrit, puis check-in, ou annulé.

Autour de ce noyau, j'ai les tables sociales : **follows_users**, qui est une **auto-référence sur users** — un utilisateur en suit un autre, donc la table pointe deux fois vers la même entité —,

follows_etablissements, **squads** et **squad_membres** en table de jonction, **invitations**, **avis**, **economies**, puis **badges** et **user_badges**. J'ai également **etablissement_photos**, qui stocke les photos de chaque lieu avec leur ordre d'affichage.

Et j'ai trois tables techniques : **user_settings** pour le thème, **login_attempts** pour la limitation du nombre de tentatives de connexion, et **password_resets** pour les jetons de réinitialisation de mot de passe.

Les contraintes d'intégrité — enchaîne directement

J'ai fait le choix de poser les garde-fous **en base**, et pas seulement en PHP, parce qu'une vérification applicative se contourne dès qu'on rejoue une requête.

Une contrainte **UNIQUE sur le couple (user_id, evenement_id)** dans inscriptions : impossible de s'inscrire deux fois au même événement, même avec un double-clic ou une requête rejouée. Côté API, je récupère d'ailleurs le **code d'erreur SQL 23000** pour renvoyer un message clair — « tu es déjà inscrit » — plutôt qu'une erreur serveur.

La même logique s'applique sur **(squad_id, user_id)**, sur **(user_id, evenement_id)** dans avis — un seul avis par personne et par événement —, et sur **(follower_id, followed_id)**.

Enfin, **toutes mes clés étrangères sont en ON DELETE CASCADE** : si un compte est supprimé, ses inscriptions, ses avis, ses follows et ses badges partent avec. C'est un choix de conception, mais c'est aussi ce qui me permet de répondre au **droit à l'effacement du RGPD** sans script de nettoyage.

3.2 — Le diagramme de cas d'utilisation (45 s)

Mon diagramme de cas d'utilisation distingue **trois acteurs** : le **visiteur**, qui ne peut que s'inscrire, se connecter et réinitialiser son mot de passe ; l'**étudiant**, avec seize cas d'usage ; et le **partenaire**, avec cinq cas centrés sur la création d'événements, le check-in et l'analytics.

Ce n'est pas un diagramme décoratif : cette séparation d'acteurs se traduit **directement dans le code** par deux fonctions de garde, **requireStudent()** et **requirePartner()**, appelées en tête de chaque page.

3.3 — Le diagramme de séquence de l'inscription (1 min)

Choisis ce scénario plutôt qu'un autre : il contient une vérification métier, une transaction et une réponse asynchrone. C'est le plus riche à commenter.

Voici le scénario le plus représentatif de l'application, l'inscription à un événement, en six étapes.

Un. L'étudiant clique sur « S'inscrire » : le JavaScript envoie une requête **POST au format JSON** sur `/api/inscrire.php`.

Deux. L'API vérifie la **session** et que le type de compte est bien « étudiant ». Cette vérification est côté serveur : je ne fais jamais confiance à ce que le front m'envoie.

Trois. Elle vérifie que l'événement **existe et n'est pas passé**, via une condition `date_heure` supérieure ou égale à maintenant.

Quatre. Elle **compte les inscrits non annulés** et compare au quota, pour refuser l'inscription si l'événement est complet.

Cinq. Elle génère le **QR code** : un hash SHA-256 construit à partir de l'identifiant utilisateur, de l'identifiant de l'événement et d'un horodatage. Il est donc unique et non devinable.

Six — et c'est le point important : l'insertion se fait dans une **transaction**. L'inscription et l'enregistrement de l'économie réalisée sont écrits ensemble, ou pas du tout. En cas d'erreur, j'appelle `rollback`, et la base reste cohérente.

Le front reçoit un JSON, déclenche une animation de confirmation et met à jour le compteur de places **sans recharger la page**.

3.4 — L'architecture applicative (1 min)

Architecturalement, j'ai retenu un **MVC allégé, procédural, sans framework**.

Le **contrôleur**, c'est le haut de chaque fichier PHP : lecture des paramètres GET et POST, validation, puis appels PDO. La **vue**, c'est le HTML en dessous, sans moteur de template. Les **traitements transverses** sont centralisés dans le dossier `includes` : `db.php` pour la connexion, `auth_check.php` pour les sessions et le contrôle d'accès, `security.php` pour la sécurité et `gamification.php` pour l'XP et les badges.

Les **actions asynchrones** passent par **douze endpoints dans le dossier /api/** qui renvoient du JSON — s'inscrire, annuler un pass, suivre un utilisateur, créer, rejoindre, quitter ou supprimer une squad, retirer un membre, inviter, lire le fil social, lire un compteur d'inscrits — auxquels s'ajoute un **treizième endpoint côté partenaire**, `api_scan.php`, qui traite le scan des QR codes.

Point clé : **toutes les pages incluent `auth_check.php` en première ligne**. Ce fichier pose les en-têtes de sécurité, démarre la session et applique le contrôle d'accès. C'est mon point d'entrée unique — ce

qui veut dire qu'aucune page ne peut être servie sans sécurité, même une page que j'ajouterais demain.

3.5 — Un problème de conception que j'ai dû résoudre (30 s)

ASTUCE — Cette anecdote fonctionne toujours : elle montre que tu identifies un problème structurel et que tu réponds par une abstraction, pas par du bricolage.

Un point que j'ai dû traiter tôt : en local sous WAMP, l'application est servie depuis le sous-dossier /TitreRNCP, alors qu'en production sur Railway elle est à la racine du domaine. Résultat, tous mes liens internes cassaient d'un environnement à l'autre.

Plutôt que de corriger chaque lien, j'ai créé une fonction **baseUrl()** côté PHP et une constante **BASE** côté JavaScript, qui détectent l'environnement à partir du nom d'hôte et préfixent les liens automatiquement.

Résultat : **le même code tourne en local et en production sans une seule ligne de configuration à changer**. C'est ce qui m'a permis d'avoir un déploiement continu réellement automatique.

3.6 — Le tour du back-end (1 min 30)

QUAND LE DIRE — Dis cette partie si tu es dans les temps à 12:00. Sinon, garde-la en réserve : c'est la réponse toute prête à la question « montre-nous comment ton code est organisé ». Le détail complet de chaque fonction est en annexes A à D.

Concrètement, tout mon back-end repose sur **quatre fichiers de socle**, dans le dossier includes, qui totalisent **31 fonctions**. Chaque page de l'application en dépend, et aucune page ne réimplémente ce qu'ils font.

db.php ouvre la connexion PDO. Il lit les identifiants dans les variables d'environnement, avec une valeur de repli pour le local, active les **exceptions** sur les erreurs SQL et le mode de récupération en tableau associatif. C'est le seul endroit du projet qui connaît les identifiants de la base.

auth_check.php est le point d'entrée obligatoire de toutes les pages. Il fixe le fuseau horaire, démarre la session, pose les en-têtes de sécurité, puis expose les fonctions de contrôle d'accès : **requireLogin**, **requireStudent** et **requirePartner**, qui redirigent selon le rôle. Il expose aussi **baseUrl**, dont j'ai parlé, et **currentUser**, qui me donne l'utilisateur courant sans requête supplémentaire, puisqu'il lit la session.

security.php contient les onze fonctions de sécurité : les en-têtes HTTP, les trois fonctions CSRF, les trois fonctions de limitation des tentatives de connexion, et le cycle complet du mot de passe oublié — création, validation et consommation du jeton.

gamification.php contient le moteur de jeu : **getUserStats** agrège en cinq requêtes les statistiques d'un utilisateur, **getXp** applique la formule de points, **getLevel** en déduit le niveau, et **checkBadges** compare ces statistiques aux conditions de chaque badge et insère ceux qui viennent d'être débloqués. Je l'appelle après chaque action qui peut faire progresser l'utilisateur.

Et un cinquième fichier, **uploads.php**, gère les photos d'établissement : il **ne fait jamais confiance au navigateur**, relit le type réel du fichier côté serveur et régénère lui-même le nom et l'extension.

Le contrat commun à tous mes endpoints

Mes treize endpoints suivent tous le même contrat, dans le même ordre : **vérifier la session et le rôle, lire et valider les paramètres, vérifier la règle métier** — le quota, la date, la propriété de la ressource —, **écrire en base**, et **répondre un JSON** contenant systématiquement un booléen de succès et un message.

Ce contrat unique est ce qui rend le code prévisible : quand j'ajoute un endpoint, je sais exactement quelles cinq étapes écrire, et je ne peux pas oublier le contrôle d'accès.

ASTUCE — Si le jury te demande d'ouvrir un fichier, ouvre `api/inscrire.php` : c'est le plus représentatif — contrôle d'accès, validation, règle métier, transaction, gestion d'erreur SQL, réponse JSON. Tout le contrat y tient en cinquante lignes.

TRANSITION → « Avant de vous montrer l'application, je veux m'arrêter sur quatre mécanismes que je considère comme les plus représentatifs de mon travail — parce qu'on me les demande systématiquement en entretien technique, et parce qu'ils justifient à eux seuls mes choix de langage. »

Partie 3.7 — Fonctionnalités clés, choix de langage et calculs

OBJECTIF — Ce sont les quatre mécanismes qu'un jury technique creuse en premier, parce qu'ils ne sont pas de simples formulaires : ils enchaînent une règle métier, une écriture en base et un effet sur d'autres fonctionnalités. Sache les dérouler sans notes.

3.7.1 — Pourquoi PHP et JavaScript « vanilla », concrètement

Ne répète pas ici l'argument déjà donné en partie 7 (« pas de framework pour un projet solo »). Apporte un niveau de preuve supplémentaire : montre ce que ce choix a changé dans l'écriture du code lui-même.

J'ai choisi le **PHP procédural typé**, sans ORM et sans moteur de template, pour une raison précise : je voulais que **chaque requête SQL soit visible et lisible telle qu'elle part vers la base**. Avec un ORM, une relation mal comprise peut générer silencieusement une requête coûteuse ; en PDO natif, ce que j'écris est ce qui s'exécute.

Concrètement, ça se traduit par deux disciplines que j'ai tenues sur tout le projet : **chaque fonction déclare le type de ses paramètres et de son retour** — c'est ce que PHPStan vérifie au niveau 5 — et **aucune page ne contient de logique dupliquée** : dès qu'un traitement sert à deux endroits, il part dans includes/. C'est ce réflexe qui a produit mes 31 fonctions de socle plutôt que du code recopié page après page.

Côté **JavaScript**, le choix du vanilla n'est pas qu'une question de poids de bundle : c'est aussi que **toutes mes interactions suivent le même patron** — un fetch en JSON, une fonction escapeHtml avant d'injecter la réponse dans le DOM, un appel à showToast pour le retour visuel. Ce patron unique, écrit une fois dans app.js, est réutilisé par les dix appels API du front. Un framework aurait imposé sa propre façon de faire ; là, c'est la mienne, et je peux l'expliquer ligne par ligne.

Et côté **SQL**, j'écris des requêtes avec des **sous-requêtes corrélées** plutôt que de charger les données en PHP puis de les recombinaisonner en boucle — c'est exactement ce qui a réglé mon problème de N+1 sur la page Explore, dont je reparle dans le bilan.

3.7.2 — La création de compte, du formulaire à la session ouverte

auth/register.php gère à la fois l'inscription étudiant et l'inscription partenaire dans un **seul formulaire**, avec un bouton qui bascule l'affichage. Voici ce qui se passe côté serveur, dans l'ordre.

1. **csrfVerify()** est appelée avant toute lecture des champs — un formulaire sans jeton valide n'est jamais traité.
2. Le **type de compte est validé par liste blanche** : seules les valeurs étudiant et partenaire sont acceptées, tout le reste retombe sur étudiant par défaut. Même chose pour le type d'établissement (bar, boîte, resto, afterwork).
3. Contrôle applicatif minimal mais réel : prénom, nom, email et mot de passe sont **obligatoires**, et le mot de passe doit faire **au moins 6 caractères**.
4. Le mot de passe est transformé avec **password_hash(\$pass, PASSWORD_DEFAULT)** — jamais stocké en clair, jamais avec un algorithme choisi à la main.

5. **INSERT dans users.** Si le compte est un partenaire et qu'un nom d'établissement a été renseigné, un **second INSERT** crée la ligne dans établissements, rattachée par user_id — dans la foulée, pas dans une étape séparée.
6. **session_regenerate_id(true)** puis remplissage de la session avec les cinq champs utiles (id, prénom, nom, type, école) — exactement les mêmes que ceux lus à la connexion, pour que currentUser() fonctionne à l'identique dans les deux cas.
7. Redirection : **onboarding.php** pour un étudiant — un tunnel de bienvenue en plusieurs écrans — et **dashboard.php** pour un partenaire, qui découvrira que son établissement est déjà créé.

ATTENTION — Un email déjà utilisé provoque une contrainte UNIQUE en base, qui remonte une erreur PDO contenant « Duplicate » — je l'intercepte pour afficher « cet email est déjà utilisé » plutôt que le message SQL brut. C'est le même réflexe que le code 23000 sur les inscriptions.

3.7.3 — La création d'un événement côté partenaire

partenaire/create_event.php illustre bien ma façon de valider : je **n'arrête pas le traitement à la première erreur**, j'accumule tous les problèmes dans un tableau \$errors, pour que le partenaire corrige tout en une seule fois plutôt qu'un champ à la fois.

- Le **titre** est obligatoire.
- Le **type** est vérifié par liste blanche (bar, boîte, resto, afterwork) — un type arbitraire envoyé depuis les outils de développement du navigateur est rejeté.
- La **date** est obligatoire.
- Le **quota** doit être d'au moins 1.
- **Si la case Flash est cochée, une date d'expiration devient elle-même obligatoire** — c'est une validation croisée entre deux champs, pas juste un contrôle isolé.

Si le tableau est vide, l'INSERT part avec les douze colonnes de la table evenements, y compris flash_expiry mis à **null explicitement** quand l'événement n'est pas flash — pour ne jamais laisser une chaîne vide interprétée comme une date par MySQL. Le partenaire est ensuite redirigé vers la liste de ses événements avec un paramètre ?created=1, qui déclenche un message de confirmation.

3.7.4 — Le système d'abonnement (follow) et son rôle transversal

ASTUCE — C'est la fonctionnalité la plus sous-estimée du projet : elle a l'air d'un simple bouton « suivre », mais elle irrigue trois autres fonctionnalités. C'est un bon exemple à donner si le jury demande « quelle partie du code est la plus interconnectée ? ».

Techniquement, suivre un étudiant est une ligne dans **follows_users**, une table qui pointe **deux fois vers users** : follower_id et followed_id. C'est ce qu'on appelle une auto-référence. L'endpoint api/follow.php reçoit une action (follow ou unfollow) et un type (user ou établissement, puisqu'on peut aussi suivre un établissement), valide les deux par liste blanche, **interdit explicitement de se suivre soi-même**, puis utilise **INSERT IGNORE** pour follow — un double clic ne provoque donc pas d'erreur — et un DELETE simple pour unfollow. Il renvoie le nouveau nombre d'abonnés pour que l'interface se mette à jour sans recharger la page.

Ce qui rend cette fonctionnalité intéressante, c'est **tout ce qui la consomme ensuite** :

- Le **fil d'activité** (api/social_feed.php) : une UNION ALL de deux requêtes qui remontent les inscriptions à des événements et les squads rejointes, **uniquement pour les personnes que je suis** — la table follows_users est jointe à inscriptions et à squad_membres.
- « **Amis inscrits** » sur un **événement** (view_event.php et la page Explore) : une jointure entre follows_users et inscriptions permet d'afficher « 3 amis y vont », avec leurs prénoms concaténés par GROUP_CONCAT.
- Le filtre « **pour moi** » sur Explore : il inclut les événements des établissements suivis et les événements où des amis sont déjà inscrits, construits comme deux listes de placeholders SQL fusionnées dans une même clause IN.
- Le **système d'invitation** (api/inviter.php) : on ne peut envoyer une invitation à un événement ou à une squad qu'à quelqu'un qu'on suit déjà — la vérification interroge directement follows_users avant d'autoriser l'INSERT dans invitations.

Une seule table, une auto-référence, et quatre fonctionnalités visibles qui en dépendent : c'est exactement ce genre d'endroit que je choisis quand on me demande de montrer « le code dont je suis le plus fier ».

3.7.5 — Le calcul de la gamification, formule par formule

J'ai déjà donné les formules dans la démonstration ; ici, je les déroule sur un **exemple chiffré**, pour prouver que je les maîtrise et pas seulement que je les récite.

Étape	Calcul	Résultat
Statistiques brutes de l'étudiant	3 événements check-in, 2 squads rejointes, 5 abonnements, 1 avis déposé	getUserStats()
XP	$3 \times 15 + 2 \times 10 + 5 \times 5 + 1 \times 8 = 45 + 20 + 25 + 8$	98 points
Niveau	$\text{floor}(\text{racine}(98 \div 50)) + 1 = \text{floor}(\text{racine}(1,96)) + 1 = \text{floor}(1,4) + 1$	Niveau 2
Seuil du niveau 3	$(3 - 1)^2 \times 50 = 4 \times 50$	200 points requis
Progression affichée	98 sur 200, soit 49 % de la barre vers le niveau 3	Barre de progression

Ce que cet exemple montre bien : la **racine carrée dans le calcul du niveau** fait qu'entre le niveau 1 et le niveau 2 il ne faut que 50 points, mais qu'entre le niveau 2 et le niveau 3 il en faut 150 de plus. La progression **ralentit mécaniquement**, sans que j'aie eu à coder un plafond ou une exception : c'est une propriété de la formule elle-même.

Et parce que **checkBadges()** est appelée juste après chaque action qui modifie ces statistiques — une inscription, un avis, un follow, une squad rejointe —, le niveau et les badges affichés au profil sont **toujours synchronisés avec la réalité**, sans tâche de fond ni recalcul périodique.

ASTUCE — Si le jury demande « pourquoi une racine carrée et pas une formule linéaire ? » : une formule linéaire donnerait le même coût à chaque niveau, ce qui n'incite pas à revenir sur l'application une fois les premiers niveaux atteints. La racine carrée récompense l'engagement initial puis demande un effort croissant — un principe classique de conception de jeu, ici implémenté en une seule ligne de PHP.

TRANSITION → « Voilà pour la conception, l'organisation du code et les mécanismes clés. Passons maintenant à l'application elle-même. »

Partie 4 — Démonstration de l'application (14:00 → 22:00)

OBJECTIF — C'est le cœur de l'oral et le moment où le jury vérifie que le projet existe vraiment. Huit minutes, un fil narratif continu, zéro hésitation sur les manipulations.

Préparation impérative avant d'entrer dans la salle

À préparer	Pourquoi
Deux sessions déjà connectées (étudiant + partenaire), côte à côte ou en deux onglets	Tu perds trente secondes et beaucoup de crédibilité si tu te trompes en tapant un mot de passe devant le jury
Un Flash Event non expiré en base de données	Le compte à rebours est l'élément le plus visuel de la démo. Un Flash expiré ruine la démonstration du concept
Un jeu de données crédible : plusieurs événements à venir, des inscrits, des avis, des squads remplies	Une application vide donne l'impression d'un prototype jamais utilisé
Une version locale prête en secours, sur le même écran	Si Railway est lent ou indisponible, tu bascules sans commenter l'incident
Des captures d'écran de tout le parcours dans le diaporama	Dernier filet de sécurité : tu continues à commenter comme si de rien n'était

ATTENTION — Ne commente jamais un incident technique (« ah, ça ne marche pas d'habitude »). Bascule sur ton plan B en continuant à parler. Le jury ne retient un problème que si tu le soulignes.

4.1 — Le parcours étudiant (3 min)

Je me mets à la place d'Arthur, qui sort de cours un jeudi après-midi.

- La page Explore** — « Voici la page principale. Chaque carte affiche le lieu, la date, la réduction, le nombre de places restantes et la note moyenne de l'établissement. En haut, cette carte avec le bandeau orange et un compte à rebours, c'est un **Flash Event** : une offre à durée limitée créée par le partenaire. Le décompte est calculé côté client à partir du champ `flash_expiry` stocké en base. »
- Les filtres** — « Je peux filtrer par type d'établissement — bar, boîte, restaurant, afterwork — et il existe un filtre « pour moi » qui remonte en priorité les événements correspondant à mon école et à mes centres d'intérêt. »
- L'inscription** — « Je clique sur « S'inscrire ». Un seul appel API, aucun rechargement de page : le compteur de places passe immédiatement de 42 à 43. Derrière, c'est exactement le diagramme de séquence que je viens de présenter : vérification de session, vérification du quota, génération du QR code et transaction. »
- Le wallet** — « Et voici mon pass, dans le wallet, avec son **QR code unique**. En haut de l'écran, le cumul de ce que j'ai économisé grâce aux réductions : il est calculé à partir du prix normal multiplié par le pourcentage de réduction, et stocké au moment de l'inscription dans la table `economies`. »
- L'historique** — « En dessous, mes pass passés, avec leur statut : check-in validé ou annulé. »

4.2 — Le parcours partenaire (2 min 30)

Bascule sur l'écran de Jean. Annonce clairement le changement de rôle pour que le jury suive.

13. **La création d'un Flash Event** — « Jean crée son événement : titre, type, date, quota, pourcentage de réduction, et la case « Flash » avec une date d'expiration. L'interface est volontairement minimale : trois champs obligatoires, parce que la cible est un gérant de bar, pas un utilisateur technique. »
14. **Le check-in** — « Le soir de l'événement, l'étudiant présente son QR code. Jean le scanne : l'inscription passe du statut « inscrit » au statut « check-in ». Deux secondes, pas de file d'attente, pas de liste papier. Et un QR déjà scanné est détecté, donc un même pass ne peut pas servir deux fois. »
15. **Le dashboard analytics** — « Et c'est ici que Jean obtient ce qu'il n'a nulle part ailleurs. Le dashboard se cale automatiquement sur son **prochain événement à venir** et affiche quatre indicateurs et deux visualisations. »
 - **Quatre compteurs** : inscrits, check-in validés, total, et **âge moyen** — déduit de la promo de chaque inscrit, L1 à M2, via une table de correspondance en PHP
 - **Une courbe Chart.js** : les inscriptions sur les **six dernières heures**, par tranche d'une heure — c'est ce qui lui permet de voir si son event décolle ou non
 - **La répartition par école** de ses inscrits, en pourcentages, calculée par un GROUP BY sur la colonne école
 - **Un compteur temps réel** rafraîchi côté client par l'endpoint /api/stats.php, qui vérifie que l'événement appartient bien au partenaire connecté avant de répondre

Concrètement, ces indicateurs lui permettent de **savoir s'il doit renforcer son équipe ce soir** et de **cibler la bonne école** pour ses prochaines offres. C'est le cas d'usage que je développe dans mon étude de cas numéro 3.

ATTENTION — Le fichier dashboard.php contient encore trois valeurs de démonstration : la variable de tendance utilise **rand()**, le conseil affiché est une **chaîne écrite en dur**, et la répartition par école bascule sur des **valeurs fictives** quand aucun inscrit n'a renseigné son école. Ne présente jamais ces trois éléments comme des calculs. Soit tu les corriges avant l'oral (voir l'annexe E), soit tu ne fais pas défiler cette zone de l'écran.

4.3 — Social, squads et gamification (2 min)

16. **Les squads** — « Je reviens côté étudiant, sur le cas de Lucas, arrivé à Clermont cette année. L'onglet Squads liste des groupes sportifs créés par les étudiants eux-mêmes : running, vélo, musculation. Il filtre par activité et par niveau, il trouve une sortie de 8 kilomètres le samedi à 18 heures, 6 membres sur 10, et il rejoint en un clic. Le quota est vérifié côté serveur, et une contrainte UNIQUE en base l'empêche de rejoindre deux fois. »
17. **Le social** — « Il peut suivre d'autres étudiants, consulter leur profil public, et retrouver leur activité dans son fil. La recherche se fait par nom, par école ou par centre d'intérêt. »
18. **La gamification** — ouvre le profil et commente la barre d'XP et la grille de badges.

Chaque action rapporte de l'expérience, selon une formule simple : **15 points par événement, 10 par squad, 5 par abonnement, 8 par avis déposé.**

Le niveau se déduit de l'XP par une **racine carrée** : partie entière de la racine de l'XP divisée par 50, plus un. C'est un choix volontaire : la progression est rapide au début, ce qui accroche le nouvel utilisateur, puis ralentit, ce qui évite que le niveau s'envole et perde son sens.

Et **huit badges se débloquent automatiquement** : la fonction `checkBadges` est appelée après chaque action majeure, elle recalcule les statistiques de l'utilisateur et insère les badges manquants, sans jamais créer de doublon.

ATTENTION — Ta base définit **9 badges**, mais `checkBadges` n'en attribue que **8** : `early bird`, « inscrit 7 jours avant un événement », existe en base mais n'est pas encore câblé. Deux options : le câbler avant l'oral, ou dire « huit badges attribués automatiquement, un neuvième déjà modélisé et prévu ». Ne dis jamais « neuf badges automatiques » : si le jury ouvre le code, l'écart coûte beaucoup plus cher que l'aveu.

4.4 — Avis, dark mode et PWA (30 s — la clôture de la démo)

Deux derniers points. **Les avis** : après un check-in validé — et seulement après —, un bouton « Laisser un avis » apparaît dans le wallet. Une note de 1 à 5 étoiles et un commentaire, uniques par utilisateur et par événement. Cette note alimente la moyenne affichée sur les cartes de la page Explore.

L'écosystème se nourrit donc de lui-même : les utilisateurs actifs rassurent les indécis.

Et deux détails d'expérience utilisateur : le **dark mode**, appliqué instantanément et conservé d'une session à l'autre, et le fait que l'application soit une **PWA installable** — je l'ajoute à mon écran d'accueil et elle s'ouvre en plein écran, sans barre de navigateur, comme une application native.

TRANSITION → « Cette application manipule des comptes, des données personnelles et des accès partenaires. Parlons donc de sécurité. »

Partie 5 — Sécurité (22:00 → 25:00)

OBJECTIF — Montrer que la sécurité n'est pas un vernis mais une architecture : annonce l'OWASP Top 10 comme grille de lecture, puis traite les risques un par un avec le nom exact de la fonction.

J'ai construit ma sécurité en suivant l'**OWASP Top 10**, et sans framework — ce qui veut dire que tout est explicite dans mon code, et que je peux justifier chaque mesure.

Risque OWASP	Ma réponse technique	La phrase à dire
Injection SQL	PDO + requêtes préparées	« Toutes les valeurs venant de l'utilisateur passent par des paramètres liés dans des requêtes préparées. Quand je construis une requête dynamiquement — le filtre de la page Explore, par exemple — je concatène uniquement des points d'interrogation , jamais des valeurs. La seule interpolation directe du projet est une clause LIMIT, sur une valeur castée en entier avant usage. »
Identification défaillante	bcrypt via password_hash(PASSWORD_DEFAULT)	« Les mots de passe sont hashés en bcrypt via password_hash avec la constante PASSWORD_DEFAULT — donc si PHP change d'algorithme par défaut, je suis automatiquement à jour. Je vérifie avec password_verify, jamais par comparaison directe. »
Fixation de session	session_regenerate_id(true)	« À chaque connexion réussie, je régénère l'identifiant de session et je détruis l'ancien. Un identifiant volé ou imposé avant la connexion devient donc inutilisable. »
CSRF	Helpers centralisés + hash_equals	Voir le développement détaillé ci-dessous — c'est le point sur lequel il faut t'attarder.
Force brute	Table login_attempts, 5 tentatives / 15 min par IP	« Au-delà de cinq tentatives en quinze minutes pour une même adresse IP, la connexion est bloquée. Les anciennes tentatives sont purgées automatiquement, et le compteur est remis à zéro après une connexion réussie. »
Réinitialisation de mot de passe	Token aléatoire, stocké en SHA-256, 1 h, usage unique	« Le jeton est un aléa de 32 octets, mais je ne stocke que son hash SHA-256 : même une fuite de base ne permettrait

Risque OWASP	Ma réponse technique	La phrase à dire
		pas de réinitialiser un compte. Il expire en une heure, ne sert qu'une fois, et les anciens jetons sont invalidés à chaque nouvelle demande. »
Énumération de comptes	Réponse identique si l'email est inconnu	« Si l'adresse est inconnue, je renvoie exactement le même message de confirmation : je ne révèle pas quels emails existent dans ma base. »
XSS	htmlspecialchars sur toute donnée utilisateur + CSP	« Tout affichage de donnée saisie par un utilisateur passe par htmlspecialchars, et ma Content-Security-Policy restreint les sources de scripts autorisées. »
Clickjacking / MIME sniffing	X-Frame-Options DENY, nosniff, HSTS, Referrer-Policy	« Une fonction setSecurityHeaders, appelée sur toutes les pages, pose l'ensemble de ces en-têtes, dont HSTS sur un an lorsqu'on est en HTTPS. »
Contrôle d'accès défaillant	requireStudent / requirePartner + vérification dans chaque endpoint	« Chaque page et chaque endpoint API vérifient la session côté serveur. Un étudiant qui appellerait /api/stats.php à la main reçoit un refus. »

Le point à développer : la protection CSRF (45 s)

ASTUCE — C'est le sujet sur lequel tu es le plus solide et le plus concret. Développe-le en trois temps : le risque, la solution, l'amélioration apportée.

Sans framework, la protection CSRF doit être posée à la main sur chaque formulaire. Le vrai risque n'est donc pas technique, il est **humain** : c'est l'oubli.

Je l'ai neutralisé en centralisant **trois fonctions dans includes/security.php**. **csrfToken()** génère un jeton de 32 octets aléatoires et le stocke en session. **csrfField()** renvoie directement le champ caché à insérer dans le formulaire — donc protéger un nouveau formulaire, c'est **une seule ligne**. Et **csrfVerify()** compare le jeton reçu avec celui de la session en utilisant **hash_equals**, et non l'opérateur d'égalité : c'est une comparaison à temps constant, qui empêche les attaques temporelles.

Et j'ai fait évoluer ce point récemment. Au départ, un jeton invalide provoquait un arrêt brutal du script : l'utilisateur tombait sur une page blanche, sans comprendre. Maintenant, je **régénère un jeton frais, je place un message d'information « session expirée » et je redirige vers la page précédente**. Le niveau de sécurité est identique, mais l'utilisateur peut se rattraper. C'est exactement le genre d'arbitrage entre sécurité et expérience utilisateur que j'ai eu à faire plusieurs fois.

TRANSITION → « Sécuriser, c'est une chose ; savoir que tout fonctionne encore après chaque modification, c'en est une autre. D'où la partie qualité. »

Partie 6 — Qualité, tests et déploiement (25:00 → 28:00)

OBJECTIF — Prouver que tu travailles comme un professionnel et pas comme un étudiant : tests automatisés, analyse statique, intégration continue, déploiement continu, documentation.

6.1 — Les tests automatisés (1 min)

J'ai **40 tests PHPUnit**, qui exécutent **282 assertions**, et l'ensemble tourne en moins de deux dixièmes de seconde. Ils se répartissent en deux familles.

Des **tests unitaires** sur les fonctions pures : la validation des entrées, le calcul de l'XP, le calcul du niveau, et les fonctions liées à l'authentification.

Et des **tests d'intégration** sur la gamification, qui tournent sur une **base SQLite en mémoire**. Cela me permet de tester de vraies requêtes SQL sans dépendre d'un serveur MySQL, donc de les exécuter en intégration continue.

C'est d'ailleurs pour cette raison que dans checkBadges je fais un SELECT suivi d'un INSERT plutôt qu'un INSERT ON DUPLICATE KEY : cette dernière syntaxe est propre à MySQL et ne fonctionnerait pas sous SQLite. **La contrainte de testabilité a influencé mon code**, et c'est assumé.

ASTUCE — Lance la suite en direct : elle prend deux dixièmes de seconde et affiche « OK (40 tests, 282 assertions) ». C'est plus convaincant que n'importe quelle diapositive.

6.2 — L'analyse statique (30 s)

En complément des tests, **PHPStan analyse le code au niveau 5** sur les dossiers includes, api et auth. Il détecte les types incohérents, les appels de méthode sur des valeurs potentiellement nulles et les paramètres mal typés — c'est-à-dire une classe entière de bugs que les tests ne verraient qu'à l'exécution.

Pour cela, j'ai **typé toutes mes signatures de fonctions et déclaré les types de retour**, ce qui rend aussi le code beaucoup plus lisible.

6.3 — L'intégration continue (45 s)

Tout cela est branché sur **GitHub Actions**. À chaque push, la CI installe les dépendances, puis rejoue l'analyse statique et l'intégralité de la suite de tests sur **trois versions de PHP : 8.1, 8.2 et 8.3**.

Je teste sur trois versions parce que je ne maîtrise pas la version installée sur l'environnement d'hébergement final. Si un test casse, le badge du README passe au rouge et je le vois immédiatement.

Concrètement, c'est ce qui me permet de **refactorer sans crainte** : j'ai retouché plusieurs fois la logique de gamification et de sécurité en cours de projet, et la CI m'a garanti à chaque fois que je ne cassais rien.

6.4 — Le déploiement continu (45 s)

Le déploiement est **continu**. Je pousse sur la branche principale, **Railway** détecte le commit, reconstruit l'image via **Nixpacks** et met en ligne en environ **deux minutes**.

Railway me fournit un **MySQL managé**, le **HTTPS automatique via Let's Encrypt** et un stockage sécurisé des variables d'environnement — mes identifiants de base de données ne sont donc **jamais** dans le dépôt Git.

Le fichier `includes/db.php` détecte l'environnement en lisant la variable `MYSQLHOST` : si elle est présente, on est sur Railway ; sinon, on est en local sous WAMP. C'est le pendant serveur de la fonction `baseUrl` dont j'ai parlé en conception.

6.5 — La documentation (15 s)

Enfin, j'ai documenté le projet comme s'il devait être repris par une équipe : un **README** d'installation, un document d'**architecture**, une documentation d'**API**, le **MCD**, les **diagrammes UML**, un **manuel utilisateur**, un rapport d'**accessibilité**, un **CHANGELOG** versionné et un fichier **SECURITY**.

C'est un réflexe que j'ai pris volontairement, même en travaillant seul : dans six mois, je serai le premier à devoir relire ce projet.

TRANSITION → « *Je termine par le bilan : ce qui a été difficile, et ce que j'en retire.* »

Partie 7 — Bilan et perspectives (28:00 → 30:00)

OBJECTIF — Montrer du recul. Un candidat qui raconte une difficulté et ce qu'il en a tiré marque plus de points qu'un candidat dont tout aurait été parfait.

7.1 — Les trois difficultés (1 min)

Raconte-les comme trois petites histoires, avec à chaque fois : le symptôme, la cause, la solution, et surtout la leçon.

Difficulté	Ce que tu racontes
La portabilité des URLs	« Mon application fonctionnait en local et cassait en production, à cause du préfixe /TitreRNCP. Plutôt que de corriger les liens un par un, j'ai créé une fonction unique baseUrl et sa constante côté JavaScript. La leçon : quand un problème revient à deux cents endroits, il ne faut pas le corriger deux cents fois, il faut créer le point de centralisation. »
La protection CSRF sans framework	« J'ai compris que le risque n'était pas technique mais humain — l'oubli d'un jeton sur un formulaire. La réponse a donc été d'en faire une seule ligne de code au lieu de dix. La leçon : la meilleure mesure de sécurité est celle qu'on ne peut pas oublier d'appliquer. »
La performance de la page Explore	« Ma première version bouclait sur les événements et lançait une requête par événement pour récupérer la note moyenne, le nombre d'inscrits et les amis présents : une quinzaine de requêtes pour afficher une seule page. C'est le problème classique du N+1 . Je l'ai résolu en intégrant des sous-requêtes corrélées directement dans la requête principale : une seule requête ramène désormais toutes les données . C'est l'optimisation dont je suis le plus satisfait, parce que j'ai mesuré la différence sur le temps de réponse. »

7.2 — Quatre choix techniques assumés (30 s)

Formule systématique : « j'ai choisi X plutôt que Y, parce que Z — et dans tel autre contexte, j'aurais choisi l'inverse. » Cette dernière moitié est ce qui distingue un choix d'un réflexe.

Choix	Justification à l'oral
PHP vanilla plutôt que Laravel ou Symfony	Sur un projet solo de cette taille, un framework aurait ajouté une couche de magie que je ne maîtrisais pas encore. Là, chaque mesure de sécurité est visible et explicable. Sur un projet d'équipe ou plus volumineux, je partirais sur Symfony pour l'ORM, le routage et les migrations.
Sessions PHP plutôt que JWT	C'est une application à rendu serveur, pas une API publique consommée par un client tiers. Les sessions sont plus simples à révoquer et évitent d'exposer un jeton côté client. Pour une application mobile native, je passerais à JWT.

Choix	Justification à l'oral
JavaScript vanilla plutôt que React	Mes interactions se résument à quelques appels API. React aurait imposé une étape de build et un bundle pour rien, sur une cible mobile où chaque kilo-octet compte au chargement.
Railway plutôt qu'un VPS	Je voulais du déploiement continu et du HTTPS sans administrer un serveur, pour concentrer mon temps sur le code. Un VPS m'aurait donné plus de contrôle, mais aussi la charge des mises à jour de sécurité système.

7.3 — Les perspectives (20 s)

Classe-les par valeur, pas par ordre technique. Cela montre que tu raisones produit.

- Les **notifications Web Push**, pour qu'un Flash Event arrive réellement en temps réel — c'est aujourd'hui le maillon faible du concept, puisque l'étudiant doit ouvrir l'application pour voir l'offre.
- Une **carte interactive** avec Leaflet et OpenStreetMap sur la page Explore, pour choisir une sortie par proximité.
- Un **score de correspondance** entre les centres d'intérêt de l'étudiant et les événements, pour personnaliser réellement le fil.
- **Stripe** pour les événements payants, et la **double authentification** sur les comptes partenaires, qui manipulent des données d'audience.

7.4 — La conclusion (20 s)

ATTENTION — Apprends cette conclusion par cœur : c'est la dernière impression que tu laisses, et c'est le moment où le stress fait le plus improviser.

Pour conclure : StudentLink part d'un besoin très simple — se retrouver entre étudiants en payant moins cher — et j'en ai fait une plateforme complète. **Dix-sept tables modélisées, treize endpoints API, une application sécurisée selon l'OWASP Top 10, quarante tests automatisés, une intégration continue sur trois versions de PHP et une mise en production réelle.**

Ce n'est pas une application fictive faite pour un examen : c'est la réponse à un problème que je vis chaque semaine, et c'est pour moi la concrétisation de mes deux années de formation.

Je vous remercie de votre attention, et je suis à votre disposition pour vos questions.

Annexe A — Le socle back-end : les 31 fonctions

Tout le projet repose sur cinq fichiers dans `includes/`. Cette annexe est ta réserve de réponses précises : tu n'as pas à la réciter, tu dois pouvoir expliquer n'importe quelle ligne si le jury ouvre un fichier.

A.1 — `includes/db.php` — la connexion (0 fonction, 1 responsabilité)

`db.php` est le **seul fichier du projet qui connaît les identifiants de la base**. Il les lit dans les variables d'environnement, avec une valeur de repli pour le développement local — c'est ce qui me permet d'avoir le même code en local et sur Railway.

- **Les cinq constantes** `DB_HOST`, `DB_NAME`, `DB_USER`, `DB_PASS` et `DB_PORT` sont définies avec ``getenv(...)`` : valeur `_locale``. En production, Railway injecte `MYSQLHOST`, `MYSQLUSER`, `MYSQLPASSWORD`, `MYSQLDATABASE` et `MYSQLPORT` ; en local, on retombe sur `localhost / root /` mot de passe vide.
- **Le DSN impose ``charset=utf8mb4``** — c'est ce qui permet de stocker correctement les accents et les emojis, y compris sur 4 octets.
- **``PDO::ATTR_ERRMODE => ERRMODE_EXCEPTION``** : toute erreur SQL lève une exception au lieu d'être silencieuse. C'est ce qui rend possible mes blocs `try/catch` et la détection du code 23000.
- **``PDO::ATTR_DEFAULT_FETCH_MODE => FETCH_ASSOC``** : je récupère des tableaux associatifs, pas des tableaux doublés en index numériques — moins de mémoire et un code plus lisible.
- **``PDO::ATTR_TIMEOUT => 10``** : au-delà de dix secondes, la tentative de connexion échoue au lieu de laisser la page pendue.

A.2 — `includes/auth_check.php` — le point d'entrée (8 fonctions)

Ce fichier est inclus en première ligne de **toutes** les pages. Avant même de déclarer ses fonctions, il exécute quatre choses : il fixe le fuseau horaire sur Europe/Paris — pour que les fenêtres de temps calculées en PHP correspondent aux horodatages écrits par MySQL —, il démarre la session si elle ne l'est pas, il charge `security.php`, et il **pose immédiatement les en-têtes de sécurité**. Autrement dit, aucune page ne peut être servie sans sécurité, même une page que j'ajouterais demain.

Fonction	Ce qu'elle fait	Comment elle fonctionne
<code>baseUrl(string \$path)</code>	Préfixe les liens internes selon l'environnement	Lit <code>HTTP_HOST</code> ; si l'hôte contient <code>localhost</code> ou <code>127.0.0.1</code> , préfixe <code>/TitreRNCP</code> , sinon chaîne vide. C'est le pendant serveur de la constante <code>BASE</code> en JavaScript.
<code>requireLogin()</code>	Interdit la page à un visiteur non connecté	Si <code>\$_SESSION['user_id']</code> est vide : redirection vers <code>auth/login.php</code> puis <code>exit</code> — l' <code>exit</code> est essentiel, sans lui la page continuerait de s'exécuter après l'en-tête.
<code>requireStudent()</code>	Réserve la page aux étudiants	Appelle <code>requireLogin</code> , puis si le type n'est pas étudiant, redirige vers le dashboard partenaire. Un partenaire n'atterrit donc pas sur une page d'erreur, mais chez lui.

Fonction	Ce qu'elle fait	Comment elle fonctionne
requirePartner()	Réserve la page aux partenaires	Même logique en miroir : un étudiant est renvoyé vers explore.php.
isLoggedIn()	Teste l'état de connexion	Retourne un booléen ; sert à afficher ou masquer des éléments d'interface, jamais à protéger une page.
currentUser()	Donne l'utilisateur courant	Construit un tableau id / prénom / nom / type / ecole à partir de la session, sans aucune requête SQL — c'est pour cela que je stocke ces cinq champs à la connexion.
themeBootScript()	Évite le flash blanc en dark mode	Retourne un script inline placé dans le <head> : il lit le thème dans localStorage et pose l'attribut data-theme sur <html> avant le premier rendu . Sans lui, la page s'afficherait en clair une fraction de seconde avant de basculer.
csrfFlash()	Affiche le message d'erreur CSRF	Récupère puis efface \$_SESSION['csrf_error'] et le rend dans une div avec role=alert et aria-live=assertive, donc annoncée par les lecteurs d'écran.

A.3 — includes/security.php — la sécurité (11 fonctions)

Les en-têtes

Fonction	Ce qu'elle fait	Comment elle fonctionne
setSecurityHeaders()	Pose sept en-têtes HTTP de sécurité	Sort immédiatement si les en-têtes sont déjà envoyés. Pose X-Frame-Options DENY, X-Content-Type-Options nosniff, X-XSS-Protection, Referrer-Policy strict-origin-when-cross-origin, HSTS d'un an uniquement si la connexion est en HTTPS , et une Content-Security-Policy qui limite les scripts à mon domaine et au CDN de Chart.js, les polices à Google Fonts, et interdit toute mise en cadre.

Les trois fonctions CSRF

Fonction	Ce qu'elle fait	Comment elle fonctionne
csrfToken()	Crée ou retourne le jeton de session	Génère bin2hex(random_bytes(32)) — soit 64 caractères hexadécimaux issus d'un générateur cryptographique — et le mémorise en session. Le même jeton sert pour toute la session.
csrfField()	Rend le champ caché du formulaire	Retourne un input hidden échappé par htmlspecialchars. Protéger un nouveau formulaire

Fonction	Ce qu'elle fait	Comment elle fonctionne
		coûte donc une seule ligne : c'est ce qui rend l'oubli improbable.
csrfVerify()	Vérifie le jeton reçu	Compare avec hash_equals , une comparaison à temps constant. En cas d'échec : supprime le jeton, pose un message flash « session expirée », redirige vers la page précédente et sort. Avant, c'était un die() qui affichait une page blanche.

Les trois fonctions anti-force brute

Fonction	Ce qu'elle fait	Comment elle fonctionne
isRateLimited(\$pdo, \$ip)	Dit si l'IP est bloquée	Purge d'abord les tentatives sorties de la fenêtre, puis compte celles des 900 dernières secondes pour cette IP et retourne vrai à partir de 5 . Les deux valeurs sont des paramètres par défaut, donc réglables sans toucher au corps de la fonction.
recordLoginAttempt()	Enregistre un échec	INSERT de l'IP et de l'email tenté — l'email sert à repérer une attaque ciblée sur un compte précis.
clearLoginAttempts()	Remet le compteur à zéro	DELETE de toutes les lignes de l'IP, appelé uniquement après une connexion réussie : un utilisateur légitime qui s'est trompé deux fois repart de zéro.

Le cycle du mot de passe oublié (4 fonctions)

Fonction	Ce qu'elle fait	Comment elle fonctionne
createPasswordResetToken()	Fabrique le jeton	Retourne null si l'email est inconnu , sans le dire à l'appelant qui affichera le même message dans tous les cas. Sinon : 32 octets aléatoires, invalidation des anciens jetons de cet email, puis stockage du SHA-256 du jeton avec une expiration à NOW() + 1 heure. Le jeton en clair n'existe que dans le lien envoyé.
validateResetToken()	Vérifie un jeton	Re-hache le jeton reçu et cherche une ligne dont le hash correspond, dont expires_at est dans le futur et dont used vaut 0. Retourne l'email associé ou null.
consumeResetToken()	Applique le nouveau mot de passe	Valide, marque le jeton used = 1 , puis met à jour le mot de passe avec password_hash.

Fonction	Ce qu'elle fait	Comment elle fonctionne
		Un même lien ne peut donc servir qu'une fois.
sendResetEmail()	Envoie le lien	En local, n'envoie rien : stocke le lien en session pour l'afficher à l'écran, ce qui rend la fonctionnalité démontrable sans serveur SMTP. En production, passe par mail().

ASTUCE — Si le jury demande « et si quelqu'un vole ta base ? » : les mots de passe sont en bcrypt et les jetons de reset sont hachés en SHA-256 — la base ne contient donc **aucun secret réutilisable**.

A.4 — includes/gamification.php — le moteur de jeu (7 fonctions)

Le principe est important à expliquer : je ne stocke **jamais** l'XP dans une colonne. Je le **recalcule** à partir des actions réellement enregistrées en base. C'est un choix : cela évite toute désynchronisation entre le compteur et la réalité — si une inscription est annulée, l'XP baisse automatiquement, sans traitement de compensation.

Fonction	Ce qu'elle fait	Comment elle fonctionne
getUserStats()	Agrège les statistiques d'un utilisateur	Cinq requêtes : nombre d'inscriptions non annulées , de squads rejointes, de personnes suivies, d'avis déposés, et somme des économies. C'est la source unique de vérité pour l'XP et les badges.
getXp(array \$stats)	Calcule les points	15 × événements + 10 × squads + 5 × follows + 8 × avis . Fonction pure, sans base de données — donc testable unitairement, et c'est le cas dans ma suite de tests.
getLevel(int \$xp)	Déduit le niveau	floor(racine(xp / 50)) + 1 , avec un minimum de 1. La courbe est volontairement racinaire : rapide au début pour accrocher, ralentie ensuite pour que le niveau garde du sens.
xpForLevel(int \$level)	Donne le seuil d'un niveau	(niveau - 1)² × 50 — c'est la réciproque de getLevel. Elle sert à dessiner la barre de progression : je connais le seuil du niveau courant et celui du suivant.
checkBadges()	Attribue les badges	Recalcule les stats, les confronte à une table de conditions (1, 5 ou 10 événements ; 1 ou 5 squads ; 1 follow ; 1 avis ; 50 € économisés), puis pour chaque condition remplie fait un SELECT puis un INSERT si le badge manque. Retourne la liste de ceux qui viennent d'être débloqués, ce qui permet d'afficher une notification.

Fonction	Ce qu'elle fait	Comment elle fonctionne
getUserBadges()	Liste les badges obtenus	JOIN entre user_badges et badges, trié par date de déblocage décroissante.
getAllBadges()	Liste tous les badges existants	Sert à afficher la grille complète, y compris les badges encore verrouillés — ce qui donne un objectif visible à l'utilisateur.

ATTENTION — Le SELECT-puis-INSERT de checkBadges n'est pas une maladresse : ``INSERT ... ON DUPLICATE KEY UPDATE`` est une syntaxe **propre à MySQL**, qui ne fonctionne pas sous SQLite — et mes tests d'intégration tournent sur SQLite en mémoire. **Dis-le comme un arbitrage assumé de testabilité**, c'est exactement ce que le jury veut entendre.

A.5 — includes/uploads.php — les images (5 fonctions)

Ce fichier applique une règle unique : **on ne fait jamais confiance au navigateur**. Ni au nom du fichier, ni au type MIME annoncé dans la requête, ni à l'extension. Tout est revérifié côté serveur, et le nom est régénéré.

Fonction	Ce qu'elle fait	Comment elle fonctionne
storeUploadedImage()	Valide et enregistre une image	Étape 1 : vérifie le code d'erreur PHP, que le fichier vient bien d'un upload (is_uploaded_file — sinon on pourrait lui faire lire un fichier système), et la taille (2 Mo). Étape 2 : lit le vrai type MIME dans le contenu avec finfo, exige jpeg, png ou webp, confirme avec getimagesize que le fichier est une image décodable, et borne les dimensions entre 200 et 6000 pixels — la borne haute est un garde-fou contre les images-bombes qui explosent la mémoire à la décompression. Étape 3 : régénère un nom de 32 caractères hexadécimaux aléatoires avec l'extension imposée par le type détecté, déplace le fichier et pose les droits en 0644.
deleteStoredImage()	Supprime un fichier	Re-valide le nom avec l'expression régulière <code>`^[a-f0-9]{32}\.(jpg png webp)\$`</code> avant de toucher au disque : c'est ce qui rend impossible une traversée de répertoire du type ../../config.
venuePhotoDir() / venuePhotoUrl()	Chemin disque et URL publique	Le chemin est calculé à partir de <code>__DIR__</code> , jamais d'une entrée utilisateur ; l'URL passe par baseUrl et rawurlencode.
uploadErrorMessage(int)	Traduit les codes d'erreur PHP	Convertit les constantes UPLOAD_ERR_* en messages lisibles en français — l'utilisateur

Fonction	Ce qu'elle fait	Comment elle fonctionne
		sait si son image est trop lourde ou si l'envoi a été coupé.

ASTUCE — C'est le fichier le plus défendable du projet en entretien technique : trois niveaux de validation, aucune confiance au client, et un nom régénéré. Si on te demande « comment tu sécuriserais un upload », tu as déjà la réponse implémentée.

Annexe B — Les 13 endpoints de l'API

Tous suivent le même contrat en cinq temps : vérifier la session et le rôle, valider les paramètres, vérifier la règle métier, écrire, répondre un JSON avec un booléen de succès et un message.

B.1 — Événements et wallet

Endpoint	Entrée	Contrôles et effet
api/inscrire.php	POST JSON { evenement_id }	Session étudiante ; l'événement doit exister et ne pas être passé ($\text{date_heure} \geq \text{NOW}()$) ; compte les inscrits non annulés et refuse si le quota est atteint ; génère le QR (SHA-256 de $\text{user_id} + \text{evenement_id} + \text{timestamp}$) ; transaction : INSERT inscription + INSERT économie si l'event a un prix et une réduction ; rollBack en cas d'erreur ; le code SQL 23000 est traduit en « tu es déjà inscrit ». Renvoie le nouveau nombre d'inscrits.
api/annuler_pass.php	POST JSON { inscription_id }	Vérifie que le pass appartient à l'utilisateur et qu'il est encore au statut inscrit — donc on ne peut pas annuler un pass déjà scanné. Transaction : passe le statut à annule et supprime la ligne d'économie correspondante, pour que le cumul reste juste.
api/stats.php	GET ?event_id	Réservé aux partenaires. Vérifie par une jointure sur établissements que l'événement appartient au partenaire connecté avant de répondre quoi que ce soit. Renvoie le nombre d'inscrits — c'est ce qui alimente le compteur temps réel du dashboard.
partenaire/api_scan.php	POST JSON { qr_code, event_id }	Vérifie la propriété de l'événement, retrouve l'inscription par le couple QR + événement — donc un QR d'un autre event est refusé —, refuse un pass déjà scanné et un pass annulé, sinon passe le statut à checkin et renvoie le prénom et le nom de l'étudiant pour affichage.

B.2 — Squads

Endpoint	Entrée	Contrôles et effet
api/create_squad.php	POST JSON (titre, type, niveau, date, lieu, quota, description)	Tronque le titre à 255 caractères et la description à 1000 ; borne le quota entre 2 et 500 ; exige un titre et une date. Transaction : crée la squad puis inscrit immédiatement le créateur comme premier

Endpoint	Entrée	Contrôles et effet
		membre — une squad n'est donc jamais vide.
api/rejoindre_squad.php	POST JSON { squad_id }	La squad doit être à venir ; compte les membres et refuse si le quota est atteint ; le code 23000 est traduit en « tu es déjà membre ». Renvoie le nouveau nombre de membres.
api/quitte_squad.php	POST JSON { squad_id }	DELETE puis test de rowCount() : s'il vaut zéro, c'est que l'utilisateur n'était pas membre — on renvoie un message précis au lieu d'un faux succès.
api/delete_squad.php	POST JSON { squad_id }	Vérifie que l'appelant est le créateur ; transaction : supprime les membres puis la squad.
api/remove_squad_member.php	POST JSON { squad_id, member_id }	Réservé au créateur ; interdit explicitement de se retirer soi-même par cette voie (il faut passer par quitter), ce qui évite qu'une squad se retrouve sans propriétaire.
api/squad_members.php	GET ?id	Réservé au créateur de la squad ; renvoie la liste des membres avec école, triée par ordre d'arrivée.

B.3 — Social

Endpoint	Entrée	Contrôles et effet
api/follow.php	POST JSON { action, type, target_id }	action vaut follow ou unfollow, type vaut user ou etablissement — les deux sont validés par liste blanche. Interdit de se suivre soi-même . Utilise INSERT IGNORE, donc un double clic ne provoque pas d'erreur. Renvoie le nombre d'abonnés mis à jour pour rafraîchir l'affichage.
api/inviter.php	POST JSON { action: send / accept / decline }	send : exige que l'expéditeur suive déjà le destinataire — on ne peut donc pas inviter un inconnu — et la contrainte d'unicité empêche le spam d'invitations. accept : passe l'invitation à accepted et inscrit automatiquement à l'événement (avec génération du QR) ou à la squad. decline : passe le statut à declined.
api/social_feed.php	GET ?limit	UNION ALL de deux requêtes : les inscriptions à des events à venir des personnes suivies, et les squads qu'elles ont rejointes. Chaque branche produit les

Endpoint	Entrée	Contrôles et effet
		mêmes colonnes avec un discriminant action_type, le tout trié par date décroissante.

ATTENTION — Question piège probable : « et le jeton CSRF sur tes endpoints JSON ? ». Réponse honnête et solide : les formulaires classiques sont protégés par csrfVerify ; les appels API sont envoyés en **Content-Type: application/json**, ce qu'un formulaire HTML d'un site tiers ne peut pas produire sans déclencher un contrôle CORS préalable — c'est la mitigation actuelle. **Et j'ajouterais un en-tête X-CSRF-Token vérifié côté serveur pour ne pas dépendre que de ça.**

Annexe C — Ce que fait chaque page côté serveur

C.1 — Authentication

Page	Traitement serveur
auth/login.php	Redirige immédiatement si l'utilisateur est déjà connecté. Sur POST : csrfVerify, puis récupération de l'IP (première valeur de X-Forwarded-For si on est derrière le proxy de Railway, sinon REMOTE_ADDR), test de rate limiting, recherche de l'utilisateur, password_verify . En cas de succès : remise à zéro des tentatives, session_regenerate_id(true) , remplissage de la session avec les cinq champs utiles, redirection selon le rôle. En cas d'échec : enregistrement de la tentative et message générique — on ne dit jamais si c'est l'email ou le mot de passe qui est faux.
auth/register.php	csrfVerify ; le type de compte est validé par liste blanche ; mot de passe d'au moins 6 caractères ; password_hash ; INSERT dans users ; si c'est un partenaire avec un nom d'établissement, INSERT dans établissements dans la foulée ; session_regenerate_id ; redirection vers l'onboarding pour un étudiant, vers le dashboard pour un partenaire. Une erreur contenant « Duplicate » est traduite en « cet email est déjà utilisé ».
auth/logout.php	Vide le tableau de session, expire explicitement le cookie de session en réutilisant ses paramètres d'origine (chemin, domaine, secure, httponly), puis session_destroy et redirection. Vider \$_SESSION seul ne suffirait pas : le cookie resterait chez le client.
auth/forgot.php	Valide le format de l'email avec filter_var, demande un jeton, tente l'envoi, puis affiche toujours le même message de confirmation — c'est la protection contre l'énumération de comptes. En local, affiche directement le lien de réinitialisation (mode développement).
auth/reset.php	Valide le jeton dès l'arrivée sur la page , avant même d'afficher le formulaire ; exige 8 caractères minimum et une confirmation identique ; appelle consumeResetToken, qui invalide le jeton dans le même mouvement.

C.2 — Pages étudiant

Page	Traitement serveur
explore.php	Deux vues dans une seule page : events et people . Vue events : une requête principale portant quatre sous-requêtes corrélées — nombre d'inscrits, inscription de l'utilisateur courant, note moyenne de l'établissement et nombre d'avis — plus une requête agrégée avec GROUP_CONCAT pour savoir quels amis sont inscrits à quel événement. Le filtre « pour moi » construit dynamiquement des placeholders (jamais des valeurs) pour les établissements suivis et les events où des amis vont. Tri : les Flash d'abord, puis par date.
view_event.php	Une requête avec deux sous-requêtes (inscrits, déjà inscrit), les photos de l'établissement triées par position, et la liste des amis inscrits. Calcule le

Page	Traitement serveur
	pourcentage de remplissage et surtout l'état flash réel : le badge n'apparaît que si flash_expiry est encore dans le futur.
wallet.php	Économies du mois et de l'année (SUM avec filtre sur MONTH et YEAR) ; pass actifs récupérés en LEFT JOIN — un choix volontaire pour que le pass reste visible même si le partenaire a supprimé l'événement, avec un indicateur event_deleted ; cinq derniers pass passés ; squads à venir.
avis.php	Refuse l'accès si l'utilisateur n'a pas le statut checkin sur cet événement — la règle métier est appliquée avant tout affichage. Note bornée entre 1 et 5, commentaire tronqué à 1000 caractères, écriture en INSERT ... ON DUPLICATE KEY UPDATE pour permettre la modification, puis appel à checkBadges.
squads.php	Une requête avec trois sous-requêtes corrélées : nombre de membres, appartenance de l'utilisateur courant, et initiales des membres concaténées pour l'affichage des avatars.
profil.php	Compteurs (inscriptions, économies, squads, abonnements, abonnés), invitations reçues via un LEFT JOIN conditionnel qui va chercher le titre dans events ou dans squads selon le type, fil d'activité en UNION ALL, puis XP, niveau et badges via gamification.php.
search_students.php et la vue people	Recherche LIKE avec échappement des caractères % et _ ; filtres par école et par centre d'intérêt (FIND_IN_SET) ; puis calcul en PHP d'un score de correspondance par intersection des centres d'intérêt, et tri par score puis par nombre de squads en commun.

C.3 — Pages partenaire

Page	Traitement serveur
partenaire/dashboard.php	Récupère l'établissement du partenaire, puis se cale automatiquement sur le prochain événement à venir . Calcule : inscrits, check-in, total, âge moyen déduit de la promo via une table de correspondance PHP (L1 → 19 ans, M2 → 23 ans...), la répartition par école par GROUP BY, et la courbe des inscriptions des six dernières heures. Intègre le scanner QR (bibliothèque html5-qrcode) qui appelle api_scan.php.
partenaire/evenements.php	Liste les événements du partenaire avec, pour chacun, le nombre d'inscrits et de check-in calculés en sous-requêtes.
partenaire/create_event.php	csrfVerify ; validation par liste blanche du type (bar, boite, resto, afterwork) ; titre et date obligatoires ; quota au minimum 1 ; et une date d'expiration obligatoire si la case Flash est cochée — sinon le compte à rebours n'aurait pas de sens. Les erreurs sont accumulées dans un tableau et affichées ensemble, pas une par une.
partenaire/edit_event.php	Même validation en modification, et affiche les statistiques de l'événement (inscrits, check-in) via des agrégats SUM conditionnels.

Page	Traitement serveur
partenaire/photos.php	Limite à 12 photos par établissement, vérifiée avant l'upload ; délègue toute la validation du fichier à storeUploadedImage ; calcule la position suivante avec $\text{MAX}(\text{position}) + 1$; à la suppression, filtre sur l'établissement du partenaire pour qu'on ne puisse pas supprimer la photo d'un autre en devinant un identifiant, et supprime le fichier sur le disque après la ligne en base.

Annexe D — Six mécanismes à savoir dérouler par cœur

Si le jury te demande « explique-moi ce qui se passe quand... », ce sont ceux-là qui tombent : l'inscription à un événement, le check-in, la réinitialisation de mot de passe, la création de compte, l'abonnement à un autre étudiant et la création d'un événement. Apprends l'enchaînement, pas les mots.

D.1 — Une inscription à un événement, de bout en bout

19. Le clic déclenche un **fetch POST en JSON** vers `/api/inscrire.php`, avec l'identifiant de l'événement.
20. La page inclut `auth_check.php`, donc la session est déjà démarrée et les en-têtes de sécurité posés.
21. L'endpoint vérifie **la session et le rôle étudiant**. S'il manque l'un des deux, il répond un JSON d'échec — pas une redirection, puisque c'est un appel asynchrone.
22. Il vérifie que **l'événement existe et n'est pas passé**, puis compte les inscriptions non annulées pour les comparer au quota.
23. Il génère le **QR code** : hash SHA-256 de l'identifiant utilisateur, de l'identifiant de l'événement et de l'horodatage.
24. Il ouvre une **transaction** : INSERT dans `inscriptions`, puis INSERT dans `economies` si l'événement a un prix et une réduction. Commit, ou rollBack si quoi que ce soit échoue.
25. Si la contrainte d'unicité saute — **code 23000** —, il répond « tu es déjà inscrit » au lieu d'une erreur serveur.
26. Le JavaScript reçoit le JSON, **échappe les données avant de les injecter dans le DOM** (fonction `escapeHtml`), met à jour le compteur et affiche une confirmation.

D.2 — Un check-in par QR code

27. Le partenaire ouvre le scanner ; la bibliothèque `html5-qrcode` lit le code et renvoie sa valeur au JavaScript.
28. Le JavaScript poste le couple `{ qr_code, event_id }` vers `partenaire/api_scan.php`.
29. L'endpoint vérifie la session partenaire, puis **que l'événement lui appartient**, par une jointure sur `etablisements`.
30. Il cherche l'inscription **par le couple QR + événement** : un pass valide pour un autre événement est donc rejeté.
31. Il **refuse un pass déjà scanné** (statut `checkin`) et un pass annulé — c'est ce qui empêche qu'un même QR serve à deux personnes.
32. Sinon il passe le statut à **checkin** et renvoie le prénom et le nom, que le partenaire voit s'afficher pour contrôle visuel.
33. Conséquence en cascade : ce statut `checkin` est ce qui **débloque le droit de laisser un avis** dans le `wallet` de l'étudiant.

D.3 — Une réinitialisation de mot de passe

34. L'utilisateur saisit son email sur `forgot.php`. Le format est validé par `filter_var`.

- 35. `createPasswordResetToken` **retourne null si l'email est inconnu** — mais la page affiche le même message de confirmation dans tous les cas.
- 36. Si l'email existe : les anciens jetons sont supprimés, un jeton de 32 octets est tiré, et **seul son SHA-256 est stocké**, avec une expiration à une heure.
- 37. Le lien contenant le jeton en clair part par email — ou s'affiche à l'écran en local, pour pouvoir démontrer la fonctionnalité sans SMTP.
- 38. Sur `reset.php`, le jeton est validé **dès l'ouverture de la page** : hash recalculé, expiration vérifiée, et champ `used` encore à 0.
- 39. À la soumission : 8 caractères minimum, confirmation identique, puis `consumeResetToken` **marque le jeton comme utilisé et remplace le mot de passe** hashé en `bcrypt`, dans la foulée.

D.4 — La création d'un compte, étudiant ou partenaire

Point de départ à expliquer : **il n'y a qu'un seul formulaire et qu'un seul traitement** pour les deux rôles. Un sélecteur bascule un champ caché « type » et fait apparaître les champs propres au rôle — école et promo pour l'étudiant, nom d'établissement, type et ville pour le partenaire. Mais c'est le même POST et le même code derrière.

- 40. La page **redirige immédiatement** si quelqu'un de déjà connecté arrive dessus : on ne crée pas un second compte par-dessus une session ouverte.
- 41. Sur POST : `csrfVerify()` en premier, avant toute lecture de données.
- 42. **Validation.** Le type est filtré par **liste blanche** — étudiant ou partenaire, avec repli sur étudiant si la valeur est trafiquée. Prénom, nom, email et mot de passe sont obligatoires. Le mot de passe doit faire **au moins 6 caractères**. Le type d'établissement est lui aussi filtré par liste blanche : bar, boîte, resto ou afterwork. La ville a une valeur par défaut, Clermont-Ferrand.
- 43. **Hachage** : `password_hash` avec `PASSWORD_DEFAULT`, donc `bcrypt`, avec un sel généré automatiquement et intégré au hash. Je ne stocke jamais le mot de passe, et je ne gère pas le sel moi-même.
- 44. **INSERT dans users.** L'école et la promo sont écrites à NULL si elles sont vides, plutôt qu'en chaîne vide — la base reste propre pour les statistiques.
- 45. **Récupération de `lastInsertId()`**, puis, si c'est un partenaire qui a saisi un nom d'établissement, **INSERT dans établissements** avec ce `user_id`. C'est le seul endroit du projet qui crée une ligne établissement — et c'est pour ça que le dashboard commence par vérifier qu'il en existe une, et redirige sinon.
- 46. **`session_regenerate_id(true)`** puis remplissage de la session avec les cinq champs utiles : identifiant, prénom, nom, type et école. C'est ce qui permet à `currentUser()` de répondre ensuite **sans aucune requête**.
- 47. **Redirection selon le rôle** : l'étudiant part sur l'onboarding, le partenaire directement sur son dashboard.
- 48. **Gestion d'erreur** : la contrainte UNIQUE sur l'email fait remonter une exception contenant « Duplicate », que je traduis en « cet email est déjà utilisé ». Toute autre erreur donne un message générique, sans détail technique.

Un mot sur **l'onboarding** : il est entièrement côté client — trois écrans que l'on fait défiler au doigt, avec un bouton « Passer ». Il **n'écrit rien en base** : c'est une explication du produit, pas une collecte.

Les centres d'intérêt sont renseignés plus tard, depuis le profil, et ils sont **validés par intersection avec une liste blanche** avant d'être enregistrés — donc impossible d'injecter une valeur arbitraire.

ATTENTION — Le bouton final de l'onboarding — et le bouton « Passer » — redirigent vers un chemin **absolu écrit en dur**, sans passer par baseUrl. En production cela fonctionne, mais **en local tu tombes sur une page 404**. Si tu démontres la création de compte étudiant sur WAMP, tu casses ta démo juste après l'inscription. C'est une correction d'une ligne (voir annexe E).

D.5 — S'abonner à un autre étudiant, et ce que ça débloque

L'abonnement est la **brique sociale centrale** : ce n'est pas une fonctionnalité isolée, c'est ce qui alimente quatre autres fonctionnalités. C'est l'angle à prendre pour l'expliquer.

Le mécanisme

49. Le bouton apparaît sur le **profil public** d'un étudiant et sur les cartes de la vue « people ». Son état initial n'est pas deviné côté client : il vient d'une **sous-requête is_following** calculée dans la requête de la page. L'affichage est donc juste dès le premier rendu, sans appel supplémentaire.
50. Au clic, le JavaScript **désactive le bouton** le temps de la requête — c'est la protection contre le double-clic — et envoie un POST JSON vers api/follow.php avec trois champs : l'action, le type de cible et son identifiant.
51. Côté serveur : session étudiante obligatoire ; **action et type validés par liste blanche** (follow ou unfollow, user ou etablissement) ; et un contrôle explicite qui **interdit de se suivre soi-même**.
52. L'écriture se fait en **INSERT IGNORE** pour un abonnement, en DELETE pour un désabonnement. INSERT IGNORE rend l'opération **idempotente** : rejouer la requête ne provoque ni doublon ni erreur, ce que garantit aussi la contrainte UNIQUE sur le couple (follower, followed).
53. La réponse renvoie le **nombre d'abonnés recompté en base**, pas une valeur incrémentée côté client : impossible que le compteur affiché dérive de la réalité.
54. Le JavaScript bascule le libellé et la couleur du bouton, et affiche une notification. Tout se fait **sans rechargement de page**.

Deux tables, un seul endpoint

Il existe **deux relations d'abonnement distinctes**. **follows_users** est une **auto-référence** : la table pointe deux fois vers users, une fois comme suiveur, une fois comme suivi. **follows_etablissements** relie un étudiant à un lieu, pour être informé de ses futures offres.

Les deux passent par le **même endpoint**, discriminé par le champ type. C'est un choix d'économie : une seule route à sécuriser et à maintenir plutôt que deux.

Ce que l'abonnement débloque ensuite — c'est ça qu'il faut dire au jury

- **Le fil d'activité** : une requête en UNION ALL qui remonte les événements auxquels les personnes suivies se sont inscrites et les squads qu'elles ont rejointes.
- **Les amis présents** affichés sur les cartes de la page Explore et sur la fiche d'un événement, via un GROUP_CONCAT des prénoms des personnes suivies qui sont inscrites.
- **Le filtre « Pour moi »** : il remonte les événements des établissements suivis **ou** ceux où des personnes suivies vont.

- **Le droit d'inviter** : `api/inviter.php` **refuse une invitation si l'expéditeur ne suit pas le destinataire**. C'est ma protection anti-spam : on ne peut pas inviter un inconnu.
- **La gamification** : chaque abonnement vaut 5 points d'XP, et le premier débloque le badge « first follow ».

ASTUCE — Formule qui marque : « L'abonnement n'est pas un bouton décoratif — c'est la donnée qui alimente le fil, les amis affichés sur les events, le filtre personnalisé et le droit d'inviter. C'est le seul endroit du produit où une action sociale a quatre conséquences fonctionnelles. »

D.6 — La création d'un événement, jusqu'à son apparition chez l'étudiant

55. La page est protégée par `requirePartner()`, puis elle récupère l'établissement du partenaire connecté. S'il n'en a pas, elle redirige — on ne peut pas créer un événement hors sol.
56. **Le formulaire** : titre, type, date et heure, description, lieu — **pré-rempli avec l'adresse de l'établissement** —, quota borné entre 1 et 10 000, prix normal, réduction en pourcentage bornée entre 0 et 100, une case « gratuit » et une case « Flash » avec sa date d'expiration.
57. **Sur POST** : `csrfVerify`, puis des **casts explicites** — entier pour le quota et la réduction, flottant pour le prix. Le type est validé par liste blanche. Les erreurs sont **accumulées dans un tableau et affichées ensemble**, pas une par une : l'utilisateur corrige tout en un passage.
58. **La règle métier propre au Flash** : si la case est cochée, la date d'expiration devient obligatoire. Sans elle, le compte à rebours n'aurait aucun sens côté étudiant.
59. **INSERT dans evenements** — et le point important : l'identifiant d'établissement vient de **la session, jamais du formulaire**. Un partenaire ne peut donc pas créer un événement au nom d'un autre établissement, même en modifiant le HTML.
60. **Redirection** vers la liste des événements avec un paramètre de confirmation. C'est le motif **POST-Redirect-GET** : si l'utilisateur rafraîchit la page, il ne recrée pas l'événement.

Comment il apparaît ensuite chez l'étudiant

61. La requête de la page Explore ne récupère que les événements dont **date_heure est supérieure ou égale à maintenant** : un événement passé disparaît tout seul, sans traitement de nettoyage.
62. Le tri est **is_flash décroissant, puis date croissante** : un Flash Event remonte automatiquement en tête de liste.
63. Le badge FLASH et le compte à rebours ne s'affichent que si **flash_expiry est encore dans le futur** — la vérification se fait à l'affichage, donc un Flash expiré redevient un événement normal sans intervention.
64. Les compteurs — inscrits, places restantes, note de l'établissement — sont des **sous-requêtes corrélées** dans la requête principale : ils sont donc toujours à jour, et sans requête supplémentaire par carte.

Modifier et supprimer

La modification passe par `edit_event.php`, avec exactement les mêmes validations. La suppression est un POST protégé par `csrfVerify`, et surtout **le DELETE est filtré sur l'identifiant de l'établissement** : un partenaire ne peut pas supprimer l'événement d'un autre en devinant un numéro.

Et une conséquence en cascade, que j'ai traitée volontairement : quand un événement est supprimé, ses inscriptions partent avec, par la contrainte ON DELETE CASCADE. C'est pour ça que le wallet de

l'étudiant récupère ses pass en **LEFT JOIN**, avec un indicateur « événement supprimé » : le pass reste visible et l'étudiant comprend ce qui s'est passé, au lieu de le voir disparaître sans explication.

Annexe E — Ce qu'il ne faut pas affirmer (et ce qu'on peut corriger avant l'oral)

POURQUOI CETTE ANNEXE — Un jury qui ouvre ton code et y trouve un écart avec ce que tu viens d'affirmer perd confiance dans tout le reste. Une limite annoncée par toi-même produit exactement l'effet inverse. Voici les écarts réels, relevés dans le code.

Écart relevé	Ce que tu dis, ou ce que tu corriges
Le dashboard contient trois valeurs de démonstration : la tendance est produite par rand(), le conseil affiché est écrit en dur, et la répartition par école bascule sur des pourcentages fictifs quand aucun inscrit n'a renseigné son école.	À corriger en priorité — c'est le seul écart qui peut se voir à l'écran pendant la démo. À défaut : ne fais pas défiler cette zone, et ne présente jamais ces éléments comme des calculs.
Le neuvième badge, early bird, est défini en base mais n'est pas attribué par checkBadges.	Soit tu le câbles (une ligne dans la table de conditions), soit tu dis « huit badges attribués automatiquement, un neuvième déjà modélisé ».
Il reste deux boucles N+1 : une requête par étudiant pour compter les squads communes dans la vue people, et six requêtes en boucle pour construire la courbe du dashboard.	Ne dis pas « j'ai éliminé tous les N+1 ». Dis : « je l'ai éliminé sur la page Explore, qui est la page la plus vue ; il en reste deux que j'ai identifiés , sur la recherche d'étudiants et sur la courbe du dashboard, et je sais comment les traiter — un GROUP BY dans le premier cas, une requête agrégée par tranche horaire dans le second. »
Une seule interpolation SQL directe : la clause LIMIT du fil social.	Elle est castée en entier avant usage, donc sûre. Mais ne dis pas « aucune concaténation dans une requête » : dis « toutes les valeurs passent en paramètres liés ; la seule interpolation est un LIMIT casté en entier ».
Les endpoints JSON ne vérifient pas de jeton CSRF (seuls les formulaires le font).	Réponse préparée : la mitigation actuelle est le Content-Type application/json, qu'un formulaire tiers ne peut pas produire sans contrôle CORS préalable — et j'ajouterais un en-tête X-CSRF-Token vérifié côté serveur.
db.php affiche le message d'erreur SQL en cas d'échec de connexion.	Correction de deux minutes : remplacer par un message générique et journaliser le détail avec error_log. À faire avant l'oral, c'est le genre de détail qu'un jury sécurité repère tout de suite.
La base compte 17 tables , pas 16 : etablissement_photos a été ajoutée avec la fonctionnalité photos.	Dis 17. Et cite la fonctionnalité photos comme une évolution postérieure au cahier des charges initial — cela montre que le projet a continué de vivre.
La fenêtre de concurrence sur le quota (lecture puis écriture sans verrou).	À annoncer toi-même comme une limite identifiée, avec la solution : SELECT ... FOR UPDATE ou un verrou sur la ligne événement.

Écart relevé	Ce que tu dis, ou ce que tu corriges
L'onboarding redirige vers un chemin absolu écrit en dur — le bouton final et le bouton « Passer » envoient vers /explore.php sans passer par baseUrl. En local, cela donne une page 404 juste après la création d'un compte étudiant.	À corriger avant l'oral, c'est une ligne : injecter <code>baseUrl('/explore.php')</code> dans le script. C'est la seule ligne du projet qui échappe encore au mécanisme baseUrl dont tu fais un argument en partie 3 — un jury attentif le remarquerait.

ASTUCE — Formulation qui marche à tous les coups : « Voilà ce que j'ai implémenté, voilà la limite que j'ai identifiée, et voilà comment je la traiterais. » Elle transforme chaque faiblesse en démonstration de recul.

Préparation aux questions du jury

Règle d'or : réponds en trois temps — la réponse directe, la justification, puis la limite que tu as identifiée. Reconnaître une limite avant que le jury ne la trouve te fait gagner des points, pas en perdre.

Questions techniques

Question	Ta réponse
Pourquoi ne pas avoir utilisé de framework ?	Projet solo de taille maîtrisée, et je voulais une sécurité explicite que je puisse justifier ligne par ligne. Sur un projet d'équipe ou plus gros, je partirais sur Symfony pour l'ORM, le routage, les migrations et la protection CSRF native.
Ton QR code est-il vraiment sécurisé ?	C'est un SHA-256 construit sur l'identifiant utilisateur, l'identifiant de l'événement et un horodatage : il n'est ni devinable ni énumérable. Il est validé côté serveur contre la table inscriptions, et le statut passe à check-in, donc un scan rejoué est détecté. La limite que j'ai identifiée : y ajouter un secret serveur en HMAC et une date d'expiration le rendrait infalsifiable même si quelqu'un connaissait ma formule.
Que se passe-t-il si deux étudiants prennent la dernière place en même temps ?	Aujourd'hui je vérifie le quota puis j'insère dans une transaction. En forte concurrence, il subsiste une fenêtre de compétition entre la lecture et l'écriture. La bonne réponse serait un SELECT FOR UPDATE, ou un verrou sur la ligne événement. C'est une limite que j'ai identifiée et que je sais corriger.
Pourquoi une seule table users pour deux rôles ?	Parce que l'authentification, la session et le profil sont communs. Les spécificités du partenaire sont déportées dans la table établissements. Cela évite de dupliquer toute la logique de connexion et simplifie les relations sociales.
Comment as-tu réglé le problème des requêtes N+1 ?	En remplaçant les boucles PHP qui relançaient une requête par événement par des sous-requêtes corrélées intégrées à la requête principale : on passe d'une quinzaine de requêtes à une seule pour afficher la page Explore.
Ton dashboard partenaire est-il cloisonné ?	Oui. L'endpoint /api/stats.php filtre sur l'établissement du partenaire connecté, jamais sur un identifiant envoyé par le client. Un partenaire ne peut donc pas lire les statistiques d'un autre.
Comment gères-tu les erreurs et les cas limites ?	Chaque endpoint renvoie un JSON avec un booléen success et un message. Les erreurs SQL sont interceptées : par exemple le code 23000, qui correspond à une violation de contrainte d'unicité, est traduit en « tu es déjà inscrit » plutôt qu'en erreur 500.
Pourquoi tester sur trois versions de PHP ?	Parce que je ne maîtrise pas la version installée sur l'environnement final. Tester de 8.1 à 8.3 garantit que le code est déployable sur la plupart des hébergeurs actuels.

Questions produit, projet et réglementaires

Question	Ta réponse
Comment passes-tu à l'échelle sur une autre ville ?	Le champ ville existe déjà sur la table établissements. Il faudrait ajouter un filtre ville sur la page Explore et un choix de ville au profil : la modélisation ne bloque pas, c'est essentiellement du travail d'interface.
Quel est le modèle économique ?	Gratuit pour l'étudiant, et un abonnement ou une commission fixe côté établissement, positionnés en dessous des plateformes existantes qui prélèvent un pourcentage sur la billetterie. La V1 sert d'abord à démontrer la valeur : remplissage et données d'audience.
Comment gères-tu le RGPD ?	Collecte minimale — pas de tracking, pas de géolocalisation. Mentions légales, CGU et politique de confidentialité en ligne. Et la suppression d'un compte entraîne, par les contraintes ON DELETE CASCADE, la suppression de toutes les données liées.
Comment as-tu validé le besoin ?	Par mon expérience directe et par des échanges avec des étudiants d'écoles différentes et un gérant d'établissement, formalisés en quatre personas. Chaque fonctionnalité du cahier des charges est rattachée à une frustration exprimée.
Comment as-tu organisé ton travail ?	En itérations, avec des conventional commits, un CHANGELOG versionné et une CI qui devait être au vert avant chaque fusion. Le cahier des charges priorisé en MoSCoW m'a servi de fil : je n'ai attaqué les Should et les Could qu'une fois tous les Must livrés.
Qu'est-ce qui a été le plus difficile ?	Techniquement, le N+1 de la page Explore. Mais le plus difficile a surtout été de m'arrêter : décider que le paiement en ligne et la messagerie n'entraient pas dans le périmètre, alors que j'avais envie de les faire.
Que ferais-tu différemment si tu recommençais ?	Je poserais la couche de tests dès le premier jour plutôt qu'en cours de route, et je centraliserais l'accès aux données dans des fonctions dédiées plutôt que d'écrire des requêtes PDO directement dans les pages. C'est ce qui me coûterait le plus cher à faire évoluer aujourd'hui.
L'application est-elle accessible ?	Je vise le niveau AA du WCAG 2.1 : navigation clavier complète, contrastes vérifiés, libellés associés aux champs de formulaire. J'ai un document dédié qui recense les points contrôlés.

ASTUCE — Si tu ne connais pas la réponse : dis-le, puis propose une piste. « Je ne l'ai pas mis en œuvre, mais je partirais sur... » vaut infiniment mieux qu'une réponse inventée, qu'un jury technique détecte immédiatement.

Glossaire — les termes que tu dois savoir définir

Un jury demande souvent de définir un terme que tu viens d'employer. Ne prononce jamais un mot que tu ne saurais pas expliquer en une phrase.

Terme	Définition en une phrase
3NF (3 ^e forme normale)	Un modèle où chaque attribut dépend de la clé, de toute la clé et de rien que la clé : cela évite les redondances et les incohérences de mise à jour.
Requête N+1	Un cas où l'on exécute une requête pour obtenir une liste, puis une requête supplémentaire par élément de cette liste — d'où un effondrement des performances.
Requête préparée	Une requête envoyée à la base avec des emplacements de paramètres, où les valeurs sont transmises séparément : les données ne peuvent donc jamais être interprétées comme du code SQL.
CSRF	Une attaque où un site tiers fait exécuter une action au nom d'un utilisateur déjà connecté ; on s'en protège avec un jeton imprévisible lié à la session.
XSS	L'injection de code JavaScript dans une page via une donnée utilisateur non échappée, exécuté ensuite dans le navigateur des autres visiteurs.
hash_equals	Une comparaison de chaînes à temps constant, qui ne s'arrête pas au premier caractère différent et empêche donc de deviner un jeton en mesurant les temps de réponse.
bcrypt	Un algorithme de hachage lent et salé, conçu pour les mots de passe : sa lenteur volontaire rend les attaques par force brute coûteuses.
Session fixation	Une attaque où l'attaquant impose un identifiant de session à la victime avant sa connexion ; on la contre en régénérant l'identifiant après authentification.
CSP (Content-Security-Policy)	Un en-tête HTTP qui déclare les sources autorisées de scripts, styles et images, et bloque tout le reste — une seconde barrière contre le XSS.
HSTS	Un en-tête qui impose au navigateur de n'utiliser que HTTPS pour le domaine pendant une durée donnée.
Transaction	Un ensemble d'opérations SQL validées ensemble ou annulées ensemble, ce qui garantit que la base ne reste jamais dans un état intermédiaire.
PWA	Une application web installable sur l'écran d'accueil, qui s'ouvre en plein écran grâce à un fichier manifest, sans passer par un store.
Intégration continue (CI)	L'exécution automatique des tests et des contrôles de qualité à chaque envoi de code, pour détecter une régression immédiatement.
MoSCoW	Une méthode de priorisation qui classe les exigences en Must, Should, Could et Won't have.

Terme	Définition en une phrase
Analyse statique	L'examen du code sans l'exécuter, pour détecter des incohérences de types ou des erreurs potentielles avant même le passage des tests.

Checklist — la veille et le jour J

La veille

- Lancer la suite de tests et vérifier qu'elle affiche **OK (40 tests, 282 assertions)**.
- Vérifier que la version en production répond et que la base contient un **jeu de données crédible**.
- Créer ou prolonger un **Flash Event non expiré** pour le jour de l'oral.
- Trancher le cas du **neuvième badge (early bird)** : le câbler, ou préparer la formulation « huit automatiques, un neuvième prévu ».
- Exporter des **captures d'écran de tout le parcours de démonstration** dans le diaporama, en secours.
- Faire une répétition complète chronométrée, à voix haute, avec la démonstration.
- Relire les tableaux « Questions du jury » et « Glossaire ».

Le jour J, avant d'entrer

- Deux sessions ouvertes (**étudiant** et **partenaire**), mots de passe déjà saisis, onglets rangés dans l'ordre de la démo.
- Notifications système coupées, mode « ne pas déranger » activé, fond d'écran neutre.
- Version locale prête en secours, sur le même écran.
- Plan minuté imprimé, posé devant toi.
- Chronomètre ou montre visible sans avoir à se contorsionner.
- Une bouteille d'eau : trente minutes de parole, c'est long.

Antisèche — la page à garder devant toi

Une seule page. Si tu ne dois emporter qu'un papier, c'est celui-ci.

Top	Partie	Les mots-clés qui relancent
00:00	Introduction	Arthur Gramond · Hesias · CDA niveau 6 · « se retrouver en payant moins cher » · annonce des 7 parties
02:00	Contexte	WhatsApp / Instagram / flyers · 42 500 Clermont · 3 012 800 France · problème symétrique · PROBLÉMATIQUE · 4 personas · Shotgun trop parisien
06:00	Cahier des charges	42 fonctionnalités · MoSCoW · 7 modules · < 2 s en 4G · 375-430 px · OWASP · WCAG AA · exclus : Stripe / natif / géoloc / messagerie
09:00	Conception	17 tables · 3NF · users / evenements / inscriptions · auto-référence follows · UNIQUE + CASCADE · 3 acteurs · séquence en 6 étapes · MVC léger · 13 endpoints · baseUrl() · 4 fichiers includes / 31 fonctions
14:00	DÉMO	Explore + Flash · inscription · wallet QR · [basculer Jean] création · scan · compteurs + courbe 6 h + écoles · squads · XP 15/10/5/8 · badges · avis · dark mode · PWA
22:00	Sécurité	OWASP · PDO préparé · bcrypt · regenerate_id · CSRF : csrfToken / csrfField / csrfVerify + hash_equals + redirection · 5 tentatives / 15 min · token SHA-256 1 h · headers · requireStudent
25:00	Qualité	40 tests / 282 assertions · SQLite en mémoire · PHPStan niveau 5 · GitHub Actions 8.1-8.2-8.3 · Railway 2 min · MySQL managé · HTTPS · 8 documents
28:00	Bilan	baseUrl · CSRF centralisé · N+1 : 15 → 1 requête · vanilla vs Laravel · sessions vs JWT · Web Push / carte / matching / Stripe / 2FA · CONCLUSION

RESPIRE — Parle plus lentement que ce qui te paraît naturel : sous stress, tout le monde accélère. Marque un silence d'une seconde après chaque titre de partie. Regarde les trois membres du jury à tour de rôle. Et souviens-toi que tu es la seule personne dans la salle à connaître ce projet par cœur.